

手写数字识别 + 雷达目标识别（在后面）

北京理工大学

仿真设计报告

报告题目 手写数字识别仿真设计报告

学 号 1120230621

姓 名 闫子易

学 科 电子科学与技术

学 院 集成电路与电子学院

2025 年 11 月 27 日

1. 手写数字识别问题背景

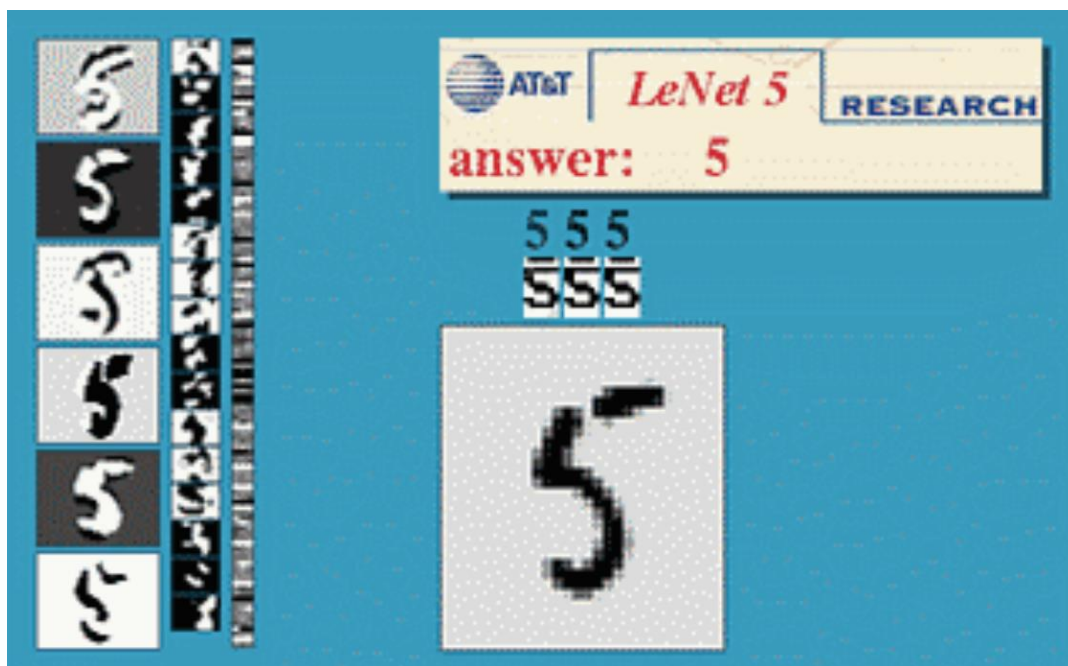
手写数字识别（HandwrittenDigitRecognition, HDR）是模式识别和计算机视觉领域中的一个基础且具有重大现实意义的问题。它旨在使机器能够像人一样，准确地理解和识别出由不同个体以各种书写风格写下的数字字符（0 到 9）。它的核心挑战源于人类书写习惯的极大差异性——即使同一人重复书写相同数字，每次的笔画、倾斜度和形状也会有所不同。此外，实际应用场景中的图像质量问题，如低对比度、笔画断裂、背景噪声和数字变形等，也进一步增加了识别难度^[1]。

根据数字来源的产生方式的不同，目前数字识别问题可以区分为手写体数字识别，印刷体数字识别，光学数字识别，自然场景下的数字识别等^[2]，具有很大的实际应用价值。例如手写体数字识别可以应用在银行汇款单号识别中，以极大的减少



人工成本，印刷体识别可以应用在邮政编码自动识别问题上，光学数字识别和自然场景数字识别则应用在车辆检测中的车牌号识别问题上^[3]。

目前比较受到关注的问题主要是手写体数字识别，由于其具有 MNIST 这种大型标准易用的成熟数据集，简单的 0-9 数字识别已经被作为计算机视觉领域的入门问题。以 LeNet-5 为例，该神经网络使用手写数字的图片作为输入，如下图中间的



方框显示，并经过神经网络的卷积层、采样层和全连接层，最终输出其对输入数字的判断，如下图 answer 处所示。

数字识别的概念是光学字符识别（OCR）的子概念^[4]，而光学字符识别起源于电报技术和为盲人创建阅读设备的技术。研究初期，识别的文字对象仅为 0-9 的数字，直至 1965 至 1970 年之间开始有一些简单的产品，数字识别技术开始被应用在邮编识别等工作场景中^[5]。1985 年，Shildhar 和 Badreldin 提出了能够准确识别手写数字的算法，他们使用拓扑特征，并结合语法分类器以高精度识别手写数字。1989 年，YannLeCun 等人在贝尔实验室将使用反向传播算法训练的卷积神经网络结合到读取“手写”数字上，并成功应用于识别美国邮政服务提供的手写邮政编码数字，成为了 LeNet 系列卷积网络的雏形^[6]。同年，YannLeCun 在发表的另一篇论文中描述了一个小的手写数字识别问题，并且表明即使该问题是线性可分的，单层网络也表现出较差的泛化能力。而当在多层的、有约束的网络上使用有位移不变性的

特征检测器时，该模型可以在此任务上表现得非常好。1990 年他们发表的论文再次描述了反向传播网络在手写数字识别中的应用，他们仅对数据进行了最小限度的预处理，而模型则是针对这项任务精心设计的，并且对其进行了高度约束。输入数据由图像组成，每张图像上包含一个数字，在美国邮政服务提供的邮政编码数字数据上的测试结果显示该模型的错误率仅有 1%，拒绝率约为 9%。

1994 年，YannLeCun，LeonBottou 等人比较了几个分类算法在手写数字标准数据库上的性能^[7]，该比较同时考虑了准确率、训练时间、识别时间等。1998 年，YannLeCun，Leon Bottou，Yoshua Bengio 和 Patrick Haffner 等人再次发表论文^[8]，回顾了应用于手写字符识别的各种方法，并用标准手写数字识别基准任务对这些模型进行了比较，结果显示卷积神经网络的表现超过了其他所有模型。该研究获得了巨大的成功，从那时起，神经网络及他们使用的 MNIST 数据集成为了手写数字识别的流行算法和验证算法的基本数据集^[9]。

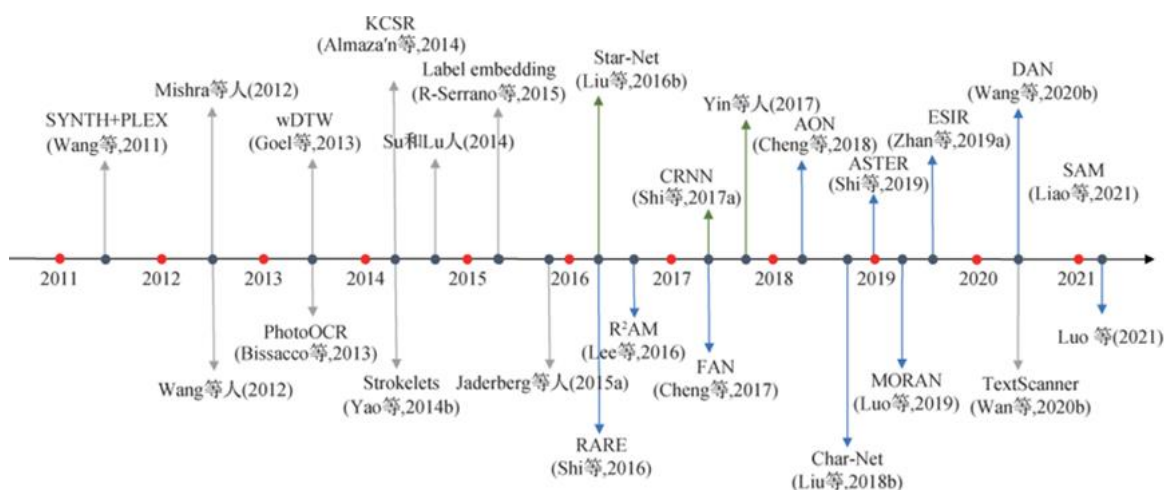
当然，神经网络并不是识别手写数字的唯一算法。早于 1997 年，Scholkopf 等人就使用支持向量机在（SVM）美国手写数字邮政服务数据库上进行了测试，测试的模型有使用 RBF 内核的 SVM、高斯核函数的 SVM 和由 SVM 方法确定的中心和由误差反向传播训练的权重的混合系统，结果显示支持向量机在当时的模型中实现了最高的精度^[10]。

手写数字识别目前已成为人工智能领域及计算机视觉领域的基本问题，大量的识别算法涌现，近几年来在 MNIST 数据集上，其识别准确率更是高达 99%^[11]。

年份	事件	相关论文/Reference
1985	Shildhar 和 Badreldin 提出了能够准确识别手写数字的算法	Shildhar M.; Badreldin A.(1985). A high accuracy syntactic recognition algorithm for handwritten numerals, IEEE T. Syst. Man Cyb. 15.

1989	Yann LeCun 等人提出了 LeNet 的最初形式	LeCun, Y.; Boser, B.; Denker, J. S.; Henderson, D.; Howard, R. E.; Hubbard, W. & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. <i>Neural Computation</i> , 1(4):541-551.
1989	Yann LeCun 分析了手写数字识别问题	LeCun, Y.(1989). Generalization and network design strategies. Technical Report CRG-TR-89-4, Department of Computer Science, University of Toronto.
1990	他们发表的论文再次描述了反向传播网络在手写数字识别中的应用	LeCun, Y.; Boser, B.; Denker, J. S.; Henderson, D.; Howard, R. E.; Hubbard, W. & Jackel, L. D. (1990). Handwritten digit recognition with a back-propagation network. <i>Advances in Neural Information Processing Systems 2 (NIPS*89)</i> .
1994	Yann LeCun, Leon Bottou 等人比较了几个分类算法在手写数字标准数据库上的性能	Bottou, L. <i>et al.</i> , (1994). Comparison of classifier methods: a case study in handwritten digit recognition. <i>Proceedings of the 12th IAPR International Conference on Pattern Recognition</i> . 3: 77-82.
1997	Scholkopf 等人就使用支持向量机在（SVM）美国手写数字邮政服务数据库上进行了测试	Scholkopf, B. <i>et al.</i> (1997). Comparing support vector machines with Gaussian kernels to radial basis function classifiers. <i>IEEE Transactions on Signal Processing</i> . 45(11): 2758-2765.
1998	他们在发表的论文中回顾了应用于手写字符识别的各种方法，并用标准手写数字识别基准任务对这些模型进行了比较，结果显示卷积神经网络的表现超过了其他所有模型	LeCun, Y.; Bottou, L.; Bengio, Y. & Haffner, P. (1998). Gradient-based learning applied to document recognition. <i>Proceedings of the IEEE</i> . 86(11): 2278 - 2324.

20 世纪 90 年代后，深度学习技术的兴起为手写数字识别带来了革命性突破，LeCun 提出的 LeNet 卷积神经网络模型，通过卷积层和池化层实现了图像特征的自动提取，将 MNIST 数据集的识别误差降至 0.7%，彻底摆脱了传统方法的特征设计局限。此后，随着 AlexNet、ResNet 等更深层网络结构的出现，手写数字识别的精度进一步提升，同时也衍生出模型轻量化、小样本学习、跨场景适配等新的研究方向，以满足移动端设备低算力、特殊场景数据稀缺等实际应用需求^[12]。



2. 手写数字识别数据集 MNIST 简介

MNIST 数据库（源自“National Institute of Standards and Technology database”）是一个通常用于训练各种数字图像处理系统的大型数据库。该数据库通过对来自 NIST 原始数据库的样本进行修改创建，涵盖手写数字的图像，共包含 60,000 张训练图像和 10,000 张测试图像，尺寸为 28×28 像素^[13]。该数据库广泛运用于机器学习领域的训练与测试当中。MNIST 在其发布时使用支持向量机的错误率为 0.8%，但一些研究后来通过使用深度学习技术显著改进了这一成绩^[14]。

上世纪 80 年代末，美国人口普查局委托 NIST 下属图像识别组评估手写表格的 OCR 能力，形成若干“特殊数据库”。受试者在含 34 个字段的手写样本表格（HSF）上书写，300dpi 扫描后切分字符。重要数据集包括 SD-1（1990）、SD-3

(1992) 和 SD-7/TD-1 (1992)。SD-1 提供切分字段，SD-3 包含 128×128 二值字符图像 (223, 125 个数字、44, 951 个大写字母、45, 313 个小写字母)，均来自 2100 名普查员；SD-7 为测试集，含 58, 646 张由马里兰一所高中 500 名学生书写的图像，附写者 ID，最初无标签，后以软盘发布，人类错误率约 1.5%。总体上 SD-3 图像更干净，SD-7 中带横划的“7”更多；早期切分器可能在构建 SD-3 时滤掉较

HANDWRITING SAMPLE FORM

NAME [REDACTED] DATE 8/23/89 CITY KERNVILLE STATE NC ZIP 27284

This sample of handwriting is being collected for use in testing computer recognition of hand printed numbers and letters. Please print the following characters in the boxes that appear below.

0123456789 0123456789 0123456789

81 095 7518 69041 938447

019 2878 25960 65533 46

9814 06100 621132 77 887

84802 070368 43 715 9937

249436 22 532 5594 17265

fmqeorvjtguilkywdxphbaas

FMQEORNVJTGUILKCYWDXPHBASZ

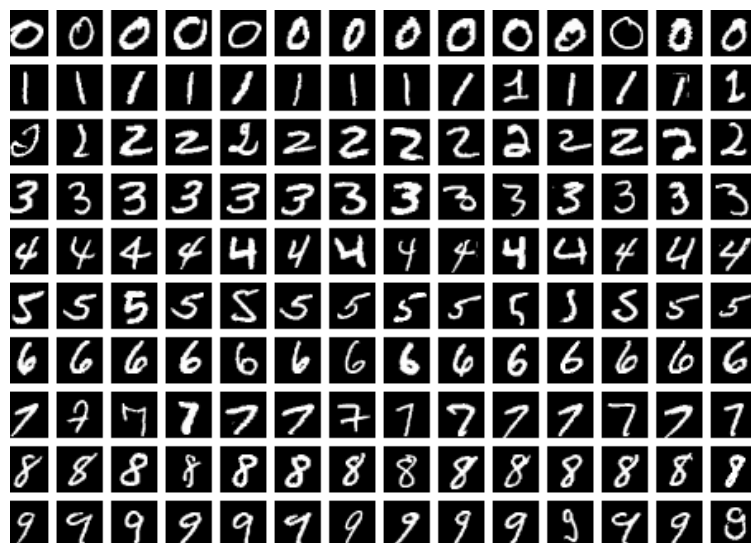
JBYNHUFQOGIZWTCLPEVXRASKD

JBYNHUFQOGIZWTCLPEVXRASKD

Please print the following text in the box below:

We, the People of the United States, in order to form a more perfect Union, establish Justice, insure domestic Tranquility, provide for the common Defense, promote the general Welfare, and secure the Blessings of Liberty to ourselves and our posterity, do ordain and establish this CONSTITUTION for the United States of America.

WE, THE PEOPLE OF THE UNITED STATES, IN ORDER TO FORM A MORE PERFECT UNION, ESTABLISH JUSTICE, INSURE DOMESTIC TRANQUILITY, PROVIDE FOR THE COMMON DEFENSE, PROMOTE THE GENERAL WELFARE, AND SECURE THE BLESSINGS OF LIBERTY TO OURSELVES AND OUR POSTERITY, DO ORDAIN AND ESTABLISH THIS CONSTITUTION FOR THE UNITED STATES OF AMERICA.



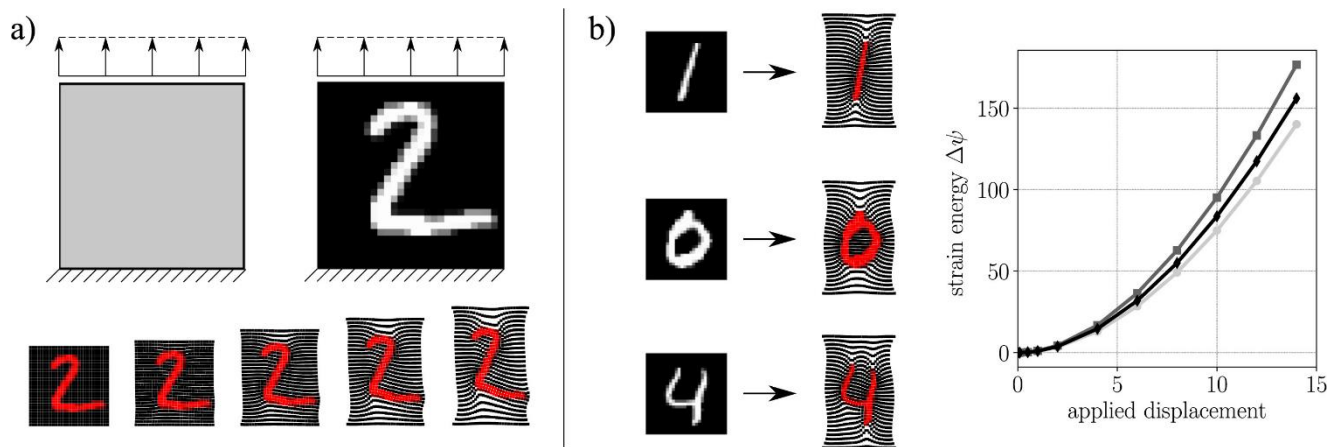
难样本，导致在 SD-3 上表现良好的系统在 SD-7 上错误率从 $<1\%$ 升至约 10% ^[15]。

1992 年 NIST 与人口普查局举办竞赛 (45 种算法参赛)，获胜者使用更大专有训练集^[16]；在使用 SD-3 的方案中，表现最好的是基于手工度量且对欧几里得变换不变的最近邻分类器。1995 年发布的 SD-19 汇编了多数据集，含 814, 255 张二值字母数字图像和 4, 169 份 HSF，2016 年更新。

MNIST 在 1994 年夏季之前完成构建。它由 SD-3 和 SD-7 中的 128×128 二值图像混合而成。具体做法是先把 SD-7 的所有图像按书写者分成训练集和测试集，每个集合各自来自 250 名书写者，因而每个集合约有 3 万张图像；随后又从 SD-3 中补充图像，直到训练集和测试集各自恰好达到 6 万张图像^[17]。

每张图像先按比例缩放，使其能放入一个 20×20 像素的框内并保持长宽比，然后进行抗锯齿处理以得到灰度图。接着通过平移把图像放入 28×28 的画布中，使像素的质心位于画布中心。下采样的具体细节后来被重建还原。

训练集和测试集最初各有 6 万个样本，但测试集中有 5 万个样本被舍弃，只保留索引号从 24476 到 34475 的那部分样本，因此最终测试集只有 1 万个样本^[18]。

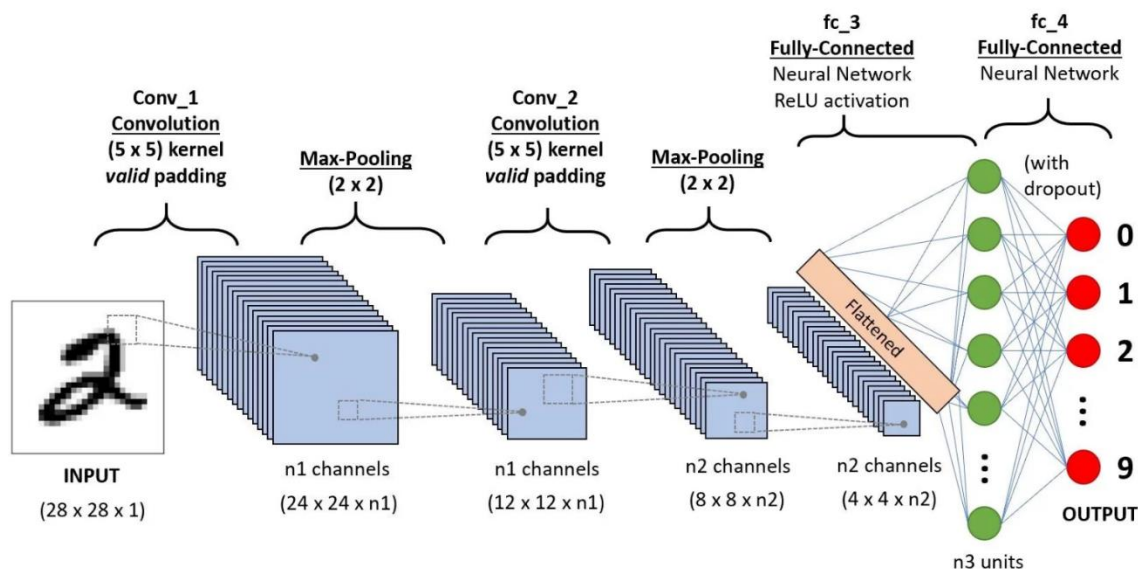


2019 年，MNIST 的完整 6 万张测试集被恢复，用以构建 QMNIST，该数据集在训练集和测试集中各包含 6 万张图像。ExtendedMNIST (EMNIST) 是 NIST 在 2017 年发布的更新数据集，被视为 MNIST 的继任者^[19]。MNIST 只包含手写数字图像，而 EMNIST 则基于 SD-19 的全部图像，按与 MNIST 相同的处理流程转换为 28×28 像素的格式。因此，原本能处理 MNIST 的工具通常无需修改就能直接用于 EMNIST。FashionMNIST 于 2017 年推出，该数据集包含 7 万张 28×28 的灰度图像，展示来自 10 个类别的时尚商品。

3. 卷积神经网络手写数字识别原理

卷积神经网络（CNN）是一类前馈神经网络，通过优化滤波器（或卷积核）来自动学习特征。这种深度学习网络已被广泛用于处理并预测多种类型的数据，包括文本、图像和音频。CNN 已成为基于深度学习的计算机视觉和图像处理方法的事实标准，直到近来在某些场景下才被诸如 Transformer 之类的新型架构部分取代^[20]。

早期神经网络在反向传播时常遇到梯度消失或梯度爆炸的问题，而卷积网络通过共享权重并减少连接数带来的正则化效应，有助于缓解这些问题。举例来说，要处理一张 100×100 像素的图像，若使用全连接层，每个神经元可能需要约 10,000 个权重；而采用级联的卷积（或互相关）核来处理 5×5 的图块时，每层卷积只需约 25 个权重。与低层特征相比，高层特征能从更宽的上下文窗口中提取信息。



卷积层

卷积层可以产生一组平行的特征图，它通过在输入图像上滑动不同的卷积核并执行一定的运算而组成。此外，在每一个滑动的位置上，卷积核与输入图像之间会执行一个元素对应乘积并求和的运算以将感受野内的信息投影到特征图中的一个元

素。这一滑动的过程可称为步幅 Z_s ，步幅 Z_s 是控制输出特征图尺寸的一个因素。卷积核的尺寸要比输入图像小得多，且重叠或平行地作用于输入图像中，一张特征图中的所有元素都是通过一个卷积核计算得出的，也即一张特征图共享了相同的权重和偏置项^[21]。

线性整流层

线性整流层 (Rectified Linear Units layer, ReLU layer) 使用线性整流 (Rectified Linear Units, ReLU) $f(x) = \max(0, x)$ 作为这一层神经的激励函数 (Activation function)。它可以增强判定函数和整个神经网络的非线性特性，而本身并不会改变卷积层。

事实上，其他的一些函数也可以用于增强网络的非线性特性，如双曲正切函数 $f(x) = \tanh(x)$ ， $f(x) = |\tanh(x)|$ ，或者 Sigmoid 函数 $f(x) = (1 + e^{-x})^{-1}$ 。相比其它函数来说，ReLU 函数更受青睐，这是因为它可以将神经网络的训练速度提升数倍，而并不会对模型的泛化准确度造成显著影响。

池化层

池化 (Pooling) 是卷积神经网络中另一个重要的概念，它实际上是一种非线性形式的降采样。有多种不同形式的非线性池化函数，而其中“最大池化 (Max pooling)”是最为常见的。它是将输入的图像划分为若干个矩形区域，对每个子区域输出最大值。

直觉上，这种机制能够有效地原因在于，一个特征的精确位置远不及它相对于其他特征的粗略位置重要。池化层会不断地减小数据的空间大小，因此参数的数量和计算量也会下降，这在一定程度上也控制了过拟合^[22]。通常来说，CNN 的网络结构中的卷积层之间都会周期性地插入池化层。池化操作提

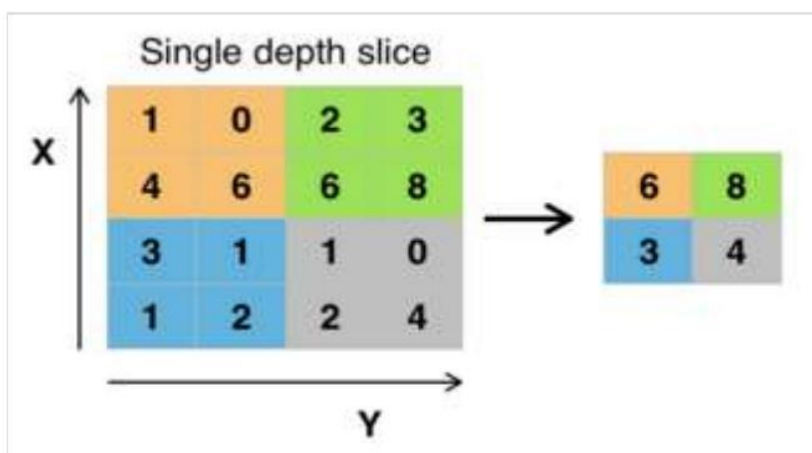
供了另一种形式的平移不变性。因为卷积核是一种特征发现器，我们通过卷积层可以很容易地发现图像中的各种边缘。但是卷积层发现的特征往往过于精确，我们即使高速连拍拍摄一个物体，照片中的物体的边缘像素位置也不大可能完全一致，通过池化层我们可以降低卷积层对边缘的敏感性。

池化层每次在一个池化窗口（depth slice）上计算输出，然后根据步幅移动池化窗口。下图是目前最常用的池化层，步幅为 2，池化窗口为 2×2 的二维最大池化层。每隔 2 个元素从图像划分出 2×2 的区块，然后对每个区块中的 4 个数取最大值。这将会减少 75% 的数据量。

$$f_{X,Y}(S) = \max_{a,b=0}^1 S_{2X+a,2Y+b}.$$

除了最大池化之外，池化层也可以使用其他池化函数，例如“平均池化”甚至“L2-范数池化”等。过去，平均池化的使用曾经较为广泛，但是最近由于最大池化在实践中的表现更好，平均池化已经不太常用。

由于池化层过快地减少了数据的大小，目前文献中的趋势是使用较小的池化滤镜，甚至不再使用池化层。



RoI 池化 (Region of Interest) 是最大池化的变体，其中输出大小是固定的，输入矩形是一个参数。

池化层是基于 Fast-RCNN 架构的卷积神经网络的一个重要组成部分。

完全连接层

最后，在经过几个卷积和最大池化层之后，神经网络中的高级推理通过完全连接层来完成^[23]。就和常规的非卷积人工神经网络中一样，完全连接层中的神经元与前一层中的所有激活都有联系。因此，它们的激活可以作为仿射变换来计算，也就是先乘以一个矩阵然后加上一个偏差 (bias) 偏移量 (向量加上一个固定的或者学习来的偏差量)。

LeNet-5 是典型的 CNN 模型，其结构如图 3-1 所示。输入为 32×32 像素的手写图像，网络共 7 层，包含 3 个卷积层、2 个下采样层、1 个全连接层与输出层。首层为卷积层 (C1)，包含 6 个 28×28 像素的特征图；下采样层 (S2) 将特征图尺寸压缩为 14×14 像素；后续卷积层 (C3) 将特征图数量扩展至 16 个，下采样层 (S4) 再次将特征图尺寸减半；末层卷积层 (C5) 包含 120 个特征图，该层等效于全连接层；全连接层 (F6) 包含 84 个单元，与 C5 的 120 个单元全连接。

最后的 Output 层也是全连接层，是 GaussianConnections，采用了 RBF 函数（即径向欧式距离函数），计算输入向量和参数向量之间的欧式距离（目前已经被 Softmax 取代）。

Output 层共有 10 个节点，分别代表数字 0 到 9。假设 x 是上一层的输入， y 是 RBF 的输出，则 RBF 输出的计算方式是：

$$y_i = \sum_{j=0}^{83} (x_j - w_{ij})^2 \quad (3-1)$$

上式中 i 取值从 0 到 9， j 取值从 0 到 $7 \times 12 - 1$ ， w 为参数。RBF 输出的值越接近于 0，则越接近于 i ，即越接近于 i 的 ASCII 编码图，表示当前网络输入的识别结果是字符 i 。

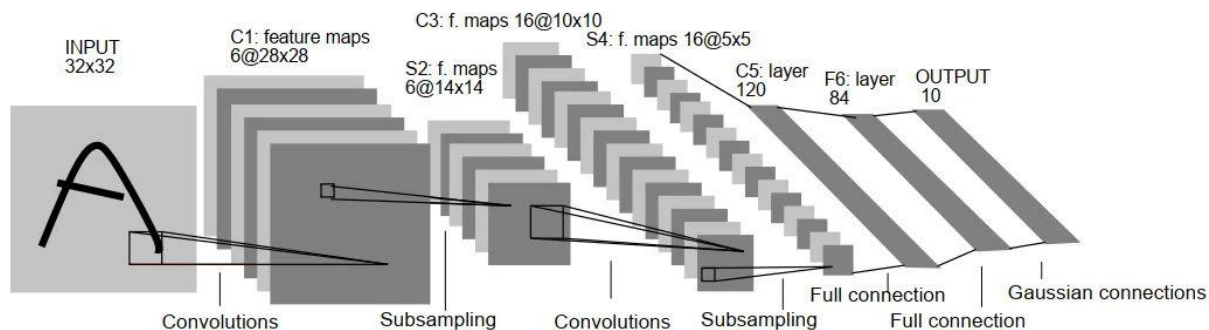


图 3.1 LeNet5 结构

图 3.2 展示了数字 3 的识别过程：

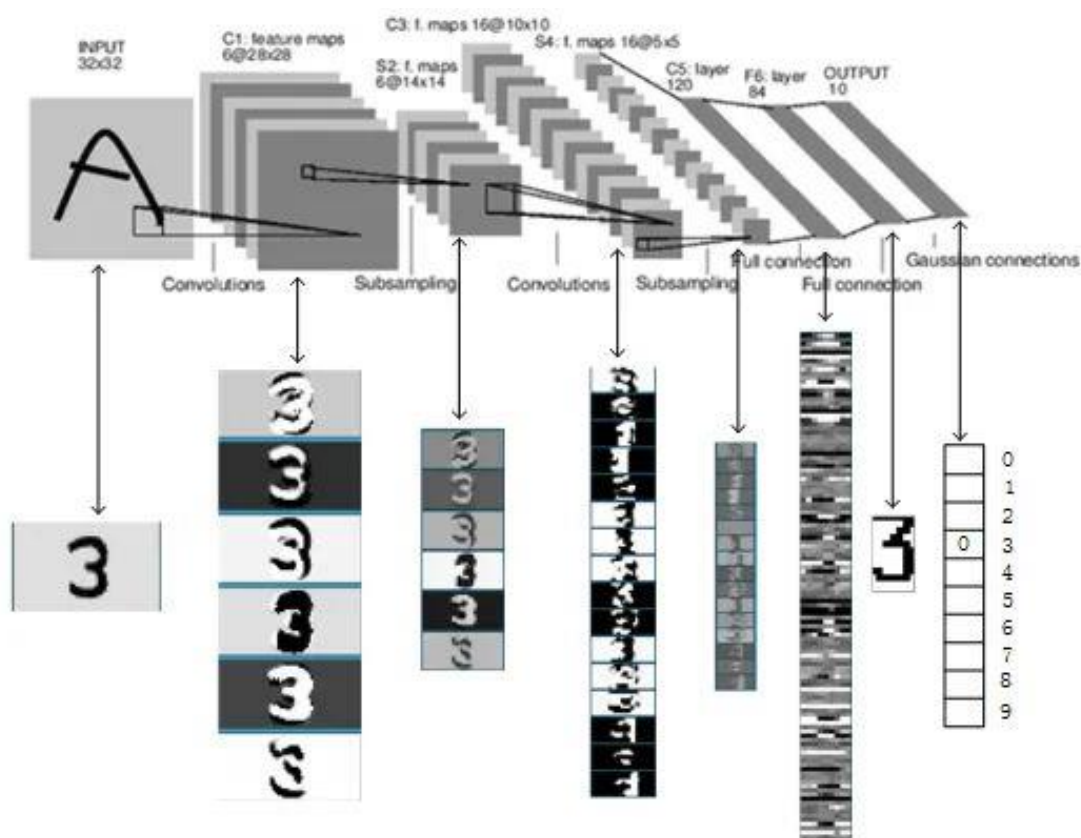


图 3.2 LeNet5 模型识别数字 3

LeNet-5 作为一种应用于手写体字符识别领域的卷积神经网络，表现出了较高的识别效率。卷积神经网络（CNN）具备提取原始图像有效特征表示的能力，这一特性使得 CNN 无需进行过多的预处理，就能直接从原始像素中识别出视觉规律。不

过，受限于当时的技术条件，一方面缺乏大规模的训练数据，另一方面计算机的计算能力也相对不足，导致 LeNet-5 在处理复杂问题时，难以取得令人满意的效果。

4. 基于 PyTorch 实现的卷积神经网络手写数字识别设计

4.1 LeNet-5 模型实现手写数字识别

我采用了完整的 3 个卷积层、2 个平均池化层以及 2 个全连接层并使用了 ReLU 激活函数。

因为 LeNet-5 模型最初是为 32×32 图像设计的，所以为了使模型更加适用于 MNIST 数据集，做出了如下调整：首先，MNIST 图像从 28×28 填充至 32×32 ，匹配 LeNet-5 原始输入要求；其次，保持单通道输入（原始 MNIST 为灰度图），无需修改模型通道数；原始 LeNet-5 最后一层卷积输出为 $120 \times 1 \times 1$ （展平 120 维），但 MNIST 经卷积后为 $120 \times 2 \times 2 = 480$ 维，需调整 fc1 输入尺寸为 480；此外，我还将原始的 Tanh/Sigmoid 替换为 ReLU，加速收敛并缓解梯度消失；最后，原始输出为欧式损失+RBF，现改为 Softmax+交叉熵损失，更适合分类任务。

在**模型构建工具与框架选择**方面，采用 PyTorch 作为主要实现框架，因其包含了动态计算图和丰富的神经网络模块，此外，使用 torch.nn 模块构建卷积层、池化层和全连接层，最后通过 torchvision 处理 MNIST 数据集加载和预处理。

在**优化器**选择与配置方面，我选择使用 Adam 优化器，因为 MNIST 图像虽然简单，但不同数字的笔画特征（如“1”的直线和“8”的复杂曲线）需要不同强度的参数更新，Adam 的自适应性能够自动处理这种差异。

在**损失函数**方面，采用了交叉熵损失（CrossEntropyLoss）以及自动整合 Softmax 激活和负对数似然计算，以便对分类任务提供稳定的梯度信号。

关键代码如下：

```
class LeNet5(torch.nn.Module):
    def __init__(self):
        super(LeNet5, self).__init__()
        # 卷积层 1: 输入 1 通道, 输出 6 通道, 5x5 卷积核, padding=2 保持尺寸不变
        self.conv1 = torch.nn.Conv2d(1, 6, kernel_size=5, stride=1,
padding=2)
        # 平均池化层 1: 2x2 窗口, 步长 2
        self.avg_pool1 = torch.nn.AvgPool2d(kernel_size=2, stride=2)
        # 卷积层 2: 输入 6 通道, 输出 16 通道, 5x5 卷积核
        self.conv2 = torch.nn.Conv2d(6, 16, kernel_size=5, stride=1)
        # 平均池化层 2: 2x2 窗口, 步长 2
        self.avg_pool2 = torch.nn.AvgPool2d(kernel_size=2, stride=2)
        # 卷积层 3: 输入 16 通道, 输出 120 通道, 5x5 卷积核
        self.conv3 = torch.nn.Conv2d(16, 120, kernel_size=5, stride=1)
        # 全连接层 1: 输入 120*2*2=480, 输出 84
        self.fc1 = torch.nn.Linear(120 * 2 * 2, 84)
        # 全连接层 2: 输入 84, 输出 10 (对应 10 个数字类别)
        self.fc2 = torch.nn.Linear(84, 10)
        # 激活函数使用 ReLU
        self.activation = torch.nn.ReLU()

    def forward(self, x):
        # 输入: 32x32x1
        x = self.activation(self.conv1(x)) # 32x32x6
        x = self.avg_pool1(x) # 16x16x6
        x = self.activation(self.conv2(x)) # 12x12x16
        x = self.avg_pool2(x) # 6x6x16
        x = self.activation(self.conv3(x)) # 2x2x120
        x = torch.flatten(x, 1) # 展平为 480 维向量(120*2*2)
        x = self.activation(self.fc1(x)) # 84 维
        x = self.fc2(x) # 10 维输出
        return x

# 创建模型实例并移动到设备
model = LeNet5().to(device)
print(model)

# 打印模型结构摘要
summary(model, (1, 32, 32))

# %% [markdown]
# ## 3. 定义损失函数和优化器
```

```

# %%
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# %% [markdown]
# ### 4. 训练和测试函数

# %%
def train(model, device, train_loader, optimizer, criterion, epoch):
    model.train()
    train_loss = 0
    correct = 0
    total = 0

    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        _, predicted = output.max(1)
        total += target.size(0)
        correct += predicted.eq(target).sum().item()

        if batch_idx % 100 == 0:
            print(f'Train Epoch: {epoch} [{batch_idx *
len(data)}/{len(train_loader.dataset)} '
                  f'({100. * batch_idx / len(train_loader):.0f}%)]\tLoss:
{loss.item():.6f}')

    train_loss /= len(train_loader)
    accuracy = 100. * correct / total
    print(f'\nTrain set: Average loss: {train_loss:.4f}, Accuracy:
{correct}/{total} ({accuracy:.2f}%)\n')
    return train_loss, accuracy

def test(model, device, test_loader, criterion):
    model.eval()
    test_loss = 0
    correct = 0

```



```

total = 0

with torch.no_grad():
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        output = model(data)
        test_loss += criterion(output, target).item()
        _, predicted = output.max(1)
        total += target.size(0)
        correct += predicted.eq(target).sum().item()

    test_loss /= len(test_loader)
    accuracy = 100. * correct / total
    print(f'Test set: Average loss: {test_loss:.4f}, Accuracy:
{correct}/{total} ({accuracy:.2f}%)\\n')
    return test_loss, accuracy

# %% [markdown]
# ## 5. 训练模型并可视化

# %%
epochs = 10
train_losses = []
train_accuracies = []
test_losses = []
test_accuracies = []

for epoch in range(1, epochs + 1):
    train_loss, train_acc = train(model, device, train_loader, optimizer,
criterion, epoch)
    test_loss, test_acc = test(model, device, test_loader, criterion)

    train_losses.append(train_loss)
    train_accuracies.append(train_acc)
    test_losses.append(test_loss)
    test_accuracies.append(test_acc)

# 可视化训练过程
plt.figure(figsize=(12, 5))

```

代码的整体思路是首先通过 torchvision 加载并预处理 MNIST 数据（调整为 32×32 尺寸以适应 LeNet-5 原始设计，进行归一化），利用数据加载器分批读取；

接着构建修正版 LeNet-5 网络结构——包含三个卷积层（使用 ReLU 激活）和两个全连接层，其中第一个卷积层添加 padding 保持尺寸，池化层采用平均池化；模型在 GPU/CPU 上训练 10 轮，使用交叉熵损失函数和 Adam 优化器，每轮记录训练/测试损失与准确率；训练完成后可视化关键指标曲线，随机抽取测试样本展示预测效果（正确/错误用颜色标注），并生成混淆矩阵分析类别级表现，并通过自定义日志模块保存控制台输出到 txt 文件，文件名标注详细的时间，方便留存和查看。

训练结果及分析

(1) 学习率 lr=0.001，迭代次数 epoch=10 时

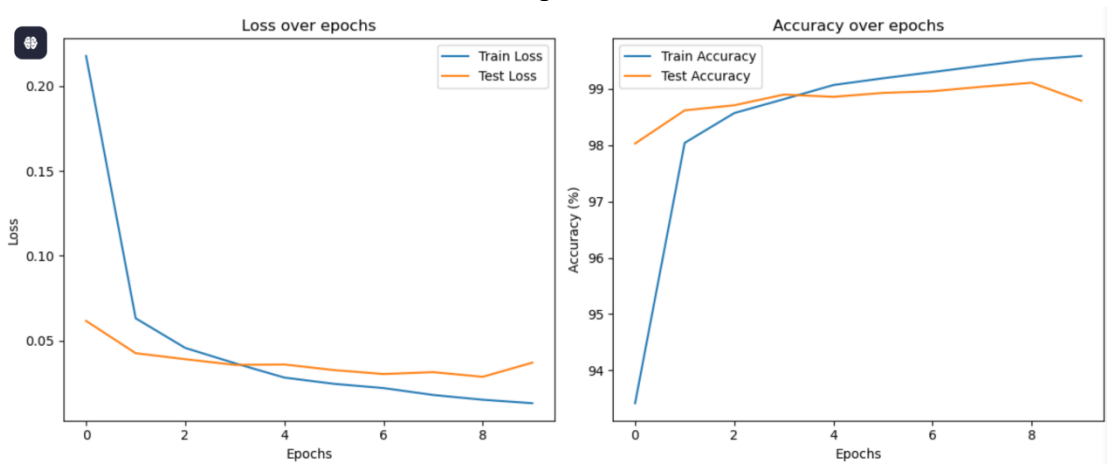


图 4.1.1 损失率及准确率随迭代次数变化函数

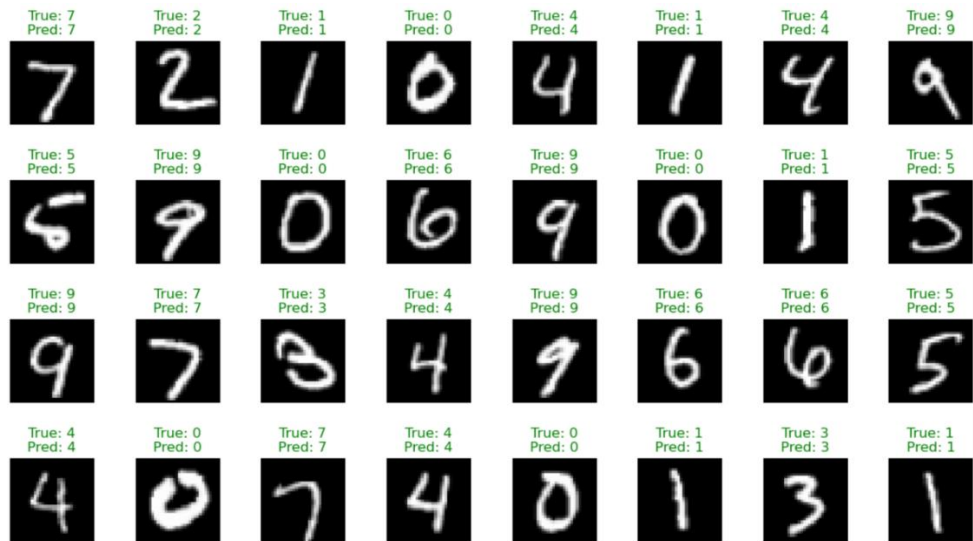


图 4.1.2 部分识别结果

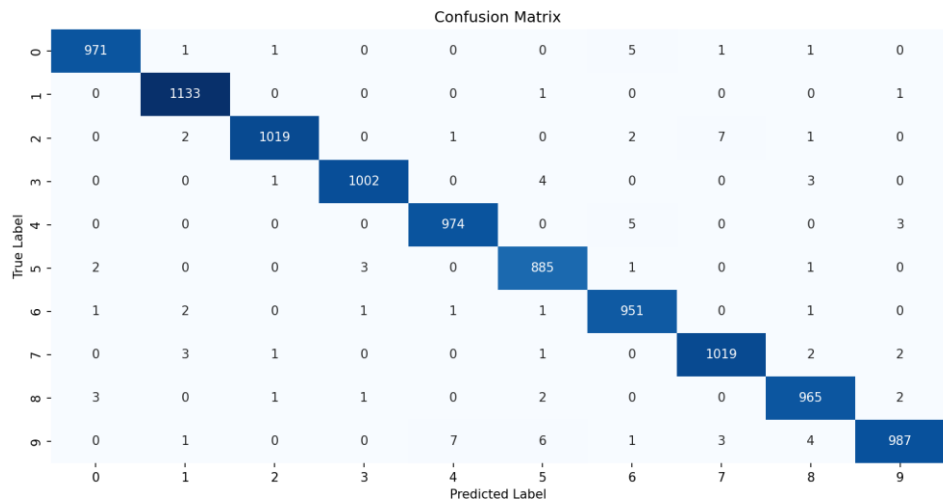


图 4.1.3 混淆矩阵

命令行输出如下：

```
PS
D:\Project\python\Intellignt_Sensing\Machine_Learning\MINST_1> &
D:/Environment/Anaconda_Env/Intelligent_Sensing_Env_2/python.exe
d:/Project/python/Intellignt_Sensing/Machine_Learning/MINST_1/LeNet_5_1.py
Using device: cuda
LeNet5
(conv1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
(avg_pool1): AvgPool2d(kernel_size=2, stride=2, padding=0)
(conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
(avg_pool2): AvgPool2d(kernel_size=2, stride=2, padding=0)
(conv3): Conv2d(16, 120, kernel_size=(5, 5), stride=(1, 1))
(fc1): Linear(in_features=480, out_features=84, bias=True)
(fc2): Linear(in_features=84, out_features=10, bias=True)
(activation): ReLU(0)
)
=====
Layer (type)          Output Shape
Param #
=====
156      Conv2d-1      [-1, 6, 32, 32]
0        ReLU-2       [-1, 6, 32, 32]
0        AvgPool2d-3  [-1, 6, 16, 16]
2,416    Conv2d-4      [-1, 16, 12, 12]
0        ReLU-5       [-1, 16, 12, 12]
0        AvgPool2d-6  [-1, 16, 6, 6]
48,120   Conv2d-7      [-1, 120, 2, 2]
0        ReLU-8       [-1, 120, 2, 2]
0        Linear-9     [-1, 84]
40,404   ReLU-10      [-1, 84]
0        Linear-11    [-1, 10]
850
=====
Total params: 91,946
Trainable params: 91,946
Non-trainable params: 0
```

```
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.15
Params size (MB): 0.35
Estimated Total Size (MB): 0.51
-----
Train Epoch: 1 [0/60000 (0%)] Loss: 2.320298
Train Epoch: 1 [6400/60000 (11%)] Loss: 0.354471
Train Epoch: 1 [12800/60000 (21%)] Loss: 0.196053
Train Epoch: 1 [19200/60000 (32%)] Loss: 0.235997
Train Epoch: 1 [25600/60000 (43%)] Loss: 0.169367
Train Epoch: 1 [32000/60000 (53%)] Loss: 0.093797
Train Epoch: 1 [38400/60000 (64%)] Loss: 0.027774
Train Epoch: 1 [44800/60000 (75%)] Loss: 0.100773
Train Epoch: 1 [51200/60000 (85%)] Loss: 0.120283
Train Epoch: 1 [57600/60000 (96%)] Loss: 0.145896
Train set: Average loss: 0.2398, Accuracy: 55666/60000 (92.78%)
Test set: Average loss: 0.0821, Accuracy: 9739/10000 (97.39%)
Train Epoch: 2 [0/60000 (0%)] Loss: 0.138031
Train Epoch: 2 [6400/60000 (11%)] Loss: 0.116672
Train Epoch: 2 [12800/60000 (21%)] Loss: 0.191895
Train Epoch: 2 [19200/60000 (32%)] Loss: 0.033881
Train Epoch: 2 [25600/60000 (43%)] Loss: 0.198370
Train Epoch: 2 [32000/60000 (53%)] Loss: 0.083756
Train Epoch: 2 [38400/60000 (64%)] Loss: 0.071465
Train Epoch: 2 [44800/60000 (75%)] Loss: 0.072719
Train Epoch: 2 [51200/60000 (85%)] Loss: 0.139915
Train Epoch: 2 [57600/60000 (96%)] Loss: 0.033755
Train set: Average loss: 0.0668, Accuracy: 58753/60000 (97.92%)
Test set: Average loss: 0.0515, Accuracy: 9827/10000 (98.27%)
```

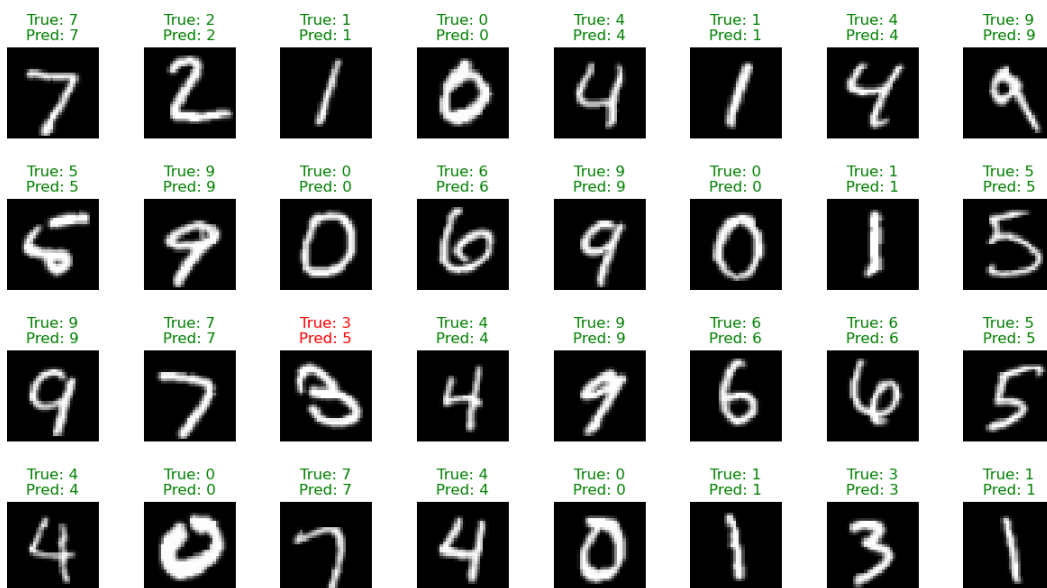
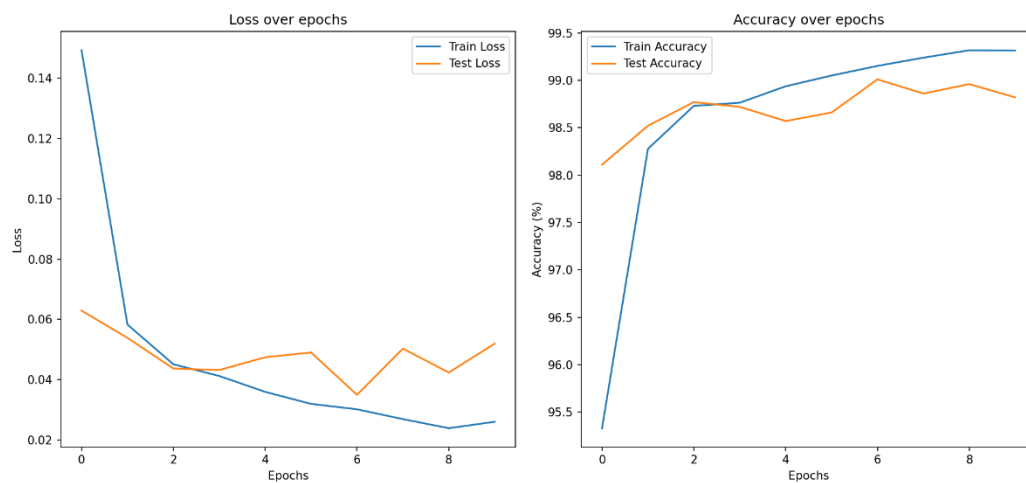
```
Train Epoch: 3 [0/60000 (0%)] Loss: 0.047910
Train Epoch: 3 [6400/60000 (11%)] Loss: 0.014719
Train Epoch: 3 [12800/60000 (21%)] Loss: 0.022811
Train Epoch: 3 [19200/60000 (32%)] Loss: 0.053083
Train Epoch: 3 [25600/60000 (43%)] Loss: 0.041599
Train Epoch: 3 [32000/60000 (53%)] Loss: 0.015077
Train Epoch: 3 [38400/60000 (64%)] Loss: 0.015541
Train Epoch: 3 [44800/60000 (75%)] Loss: 0.009021
Train Epoch: 3 [51200/60000 (85%)] Loss: 0.010846
Train Epoch: 3 [57600/60000 (96%)] Loss: 0.008761
Train set: Average loss: 0.0457, Accuracy: 59176/60000 (98.63%)
Test set: Average loss: 0.0381, Accuracy: 9872/10000 (98.72%)
Train Epoch: 4 [0/60000 (0%)] Loss: 0.064493
Train Epoch: 4 [6400/60000 (11%)] Loss: 0.004774
Train Epoch: 4 [12800/60000 (21%)] Loss: 0.051594
Train Epoch: 4 [19200/60000 (32%)] Loss: 0.180765
Train Epoch: 4 [25600/60000 (43%)] Loss: 0.003864
Train Epoch: 4 [32000/60000 (53%)] Loss: 0.059149
Train Epoch: 4 [38400/60000 (64%)] Loss: 0.026333
Train Epoch: 4 [44800/60000 (75%)] Loss: 0.043283
Train Epoch: 4 [51200/60000 (85%)] Loss: 0.024720
Train Epoch: 4 [57600/60000 (96%)] Loss: 0.011804
Train set: Average loss: 0.0353, Accuracy: 59332/60000 (98.89%)
Test set: Average loss: 0.0366, Accuracy: 9888/10000 (98.88%)
Train Epoch: 5 [0/60000 (0%)] Loss: 0.103013
Train Epoch: 5 [6400/60000 (11%)] Loss: 0.002085
Train Epoch: 5 [12800/60000 (21%)] Loss: 0.126667
```

Train Epoch: 5 [19200/60000 (32%)]	Loss: 0.028711	Train Epoch: 7 [12800/60000 (21%)]	Loss: 0.005235	Train Epoch: 9 [6400/60000 (11%)]	Loss: 0.009526
Train Epoch: 5 [25600/60000 (43%)]	Loss: 0.016612	Train Epoch: 7 [19200/60000 (32%)]	Loss: 0.001814	Train Epoch: 9 [12800/60000 (21%)]	Loss: 0.000228
Train Epoch: 5 [32000/60000 (53%)]	Loss: 0.002621	Train Epoch: 7 [25600/60000 (43%)]	Loss: 0.010469	Train Epoch: 9 [19200/60000 (32%)]	Loss: 0.006684
Train Epoch: 5 [38400/60000 (64%)]	Loss: 0.027162	Train Epoch: 7 [32000/60000 (53%)]	Loss: 0.029167	Train Epoch: 9 [25600/60000 (43%)]	Loss: 0.000338
Train Epoch: 5 [44800/60000 (75%)]	Loss: 0.020345	Train Epoch: 7 [38400/60000 (64%)]	Loss: 0.017179	Train Epoch: 9 [32000/60000 (53%)]	Loss: 0.000069
Train Epoch: 5 [51200/60000 (85%)]	Loss: 0.013735	Train Epoch: 7 [44800/60000 (75%)]	Loss: 0.056482	Train Epoch: 9 [38400/60000 (64%)]	Loss: 0.000525
Train Epoch: 5 [57600/60000 (96%)]	Loss: 0.072201	Train Epoch: 7 [51200/60000 (85%)]	Loss: 0.004576	Train Epoch: 9 [44800/60000 (75%)]	Loss: 0.013062
Train set: Average loss: 0.0288, Accuracy: 59462/60000 (99.10%)		Train Epoch: 7 [57600/60000 (96%)]	Loss: 0.000959	Train Epoch: 9 [51200/60000 (85%)]	Loss: 0.000336
Test set: Average loss: 0.0285, Accuracy: 9902/10000 (99.02%)		Train set: Average loss: 0.0194, Accuracy: 59611/60000 (99.35%)		Train Epoch: 9 [57600/60000 (96%)]	Loss: 0.016264
Train Epoch: 6 [0/60000 (0%)]	Loss: 0.008449	Test set: Average loss: 0.0364, Accuracy: 9888/10000 (98.88%)		Train set: Average loss: 0.0145, Accuracy: 59728/60000 (99.55%)	
Train Epoch: 6 [6400/60000 (11%)]	Loss: 0.000318	Train Epoch: 8 [0/60000 (0%)]	Loss: 0.017484	Test set: Average loss: 0.0356, Accuracy: 9900/10000 (99.00%)	
Train Epoch: 6 [12800/60000 (21%)]	Loss: 0.004918	Train Epoch: 8 [6400/60000 (11%)]	Loss: 0.011793	Train Epoch: 10 [0/60000 (0%)]	Loss: 0.024001
Train Epoch: 6 [19200/60000 (32%)]	Loss: 0.083711	Train Epoch: 8 [12800/60000 (21%)]	Loss: 0.003738	Train Epoch: 10 [6400/60000 (11%)]	Loss: 0.000544
Train Epoch: 6 [25600/60000 (43%)]	Loss: 0.024514	Train Epoch: 8 [19200/60000 (32%)]	Loss: 0.004557	Train Epoch: 10 [12800/60000 (21%)]	Loss: 0.000431
Train Epoch: 6 [32000/60000 (53%)]	Loss: 0.021864	Train Epoch: 8 [25600/60000 (43%)]	Loss: 0.002632	Train Epoch: 10 [19200/60000 (32%)]	Loss: 0.018990
Train Epoch: 6 [38400/60000 (64%)]	Loss: 0.022131	Train Epoch: 8 [32000/60000 (53%)]	Loss: 0.022082	Train Epoch: 10 [25600/60000 (43%)]	Loss: 0.004328
Train Epoch: 6 [44800/60000 (75%)]	Loss: 0.032491	Train Epoch: 8 [38400/60000 (64%)]	Loss: 0.018585	Train Epoch: 10 [32000/60000 (53%)]	Loss: 0.014579
Train Epoch: 6 [51200/60000 (85%)]	Loss: 0.010058	Train Epoch: 8 [44800/60000 (75%)]	Loss: 0.002580	Train Epoch: 10 [38400/60000 (64%)]	Loss: 0.060362
Train Epoch: 6 [57600/60000 (96%)]	Loss: 0.007716	Train Epoch: 8 [51200/60000 (85%)]	Loss: 0.019610	Train Epoch: 10 [44800/60000 (75%)]	Loss: 0.000778
Train set: Average loss: 0.0238, Accuracy: 59530/60000 (99.22%)		Train Epoch: 8 [57600/60000 (96%)]	Loss: 0.014252	Train Epoch: 10 [51200/60000 (85%)]	Loss: 0.023186
Test set: Average loss: 0.0313, Accuracy: 9902/10000 (99.02%)		Train set: Average loss: 0.0189, Accuracy: 59642/60000 (99.40%)		Train Epoch: 10 [57600/60000 (96%)]	Loss: 0.013818
Train Epoch: 7 [0/60000 (0%)]	Loss: 0.145355	Test set: Average loss: 0.0333, Accuracy: 9905/10000 (99.05%)		Train set: Average loss: 0.0139, Accuracy: 59740/60000 (99.57%)	
Train Epoch: 7 [6400/60000 (11%)]	Loss: 0.003711	Train Epoch: 9 [0/60000 (0%)]	Loss: 0.000556	Test set: Average loss: 0.0320, Accuracy: 9906/10000 (99.06%)	

整体来看，这次用改良的 LeNet-5 在 MNIST 上的训练非常成功：模型参数量小（约 9.2 万），训练在 GPU 上完成，10 个 epoch 后训练集准确率接近 **99.6%**，测试集准确率稳定在 **99.0%左右**，测试集平均损失降到约 **0.03**。从损失和准确率曲线可以看到训练过程收敛迅速，前几轮就已达较高性能，随后训练损失和测试损失都继续缓慢下降，训练准确率略高于测试准确率但差距很小，说明模型既能很好拟合训练数据，又具有较好的泛化能力。

进一步看混淆矩阵，绝大多数类别的对角线值非常高，说明模型对大部分数字识别几乎无误。少数错误集中在特定类别之间：例如 0 与 6、8 之间有少量混淆，8 与 3/9/0 有少量互相误判，9 偶尔被判为 4、5 或 8，5 在某些样本上也有被误判为 0 或 3 的情况。这类错误多为笔画相近或书写不规范导致的局部相似性问题，属于常见的手写数字识别误差。总体误差分布并不集中在某一类，说明模型没有明显的类别偏置。

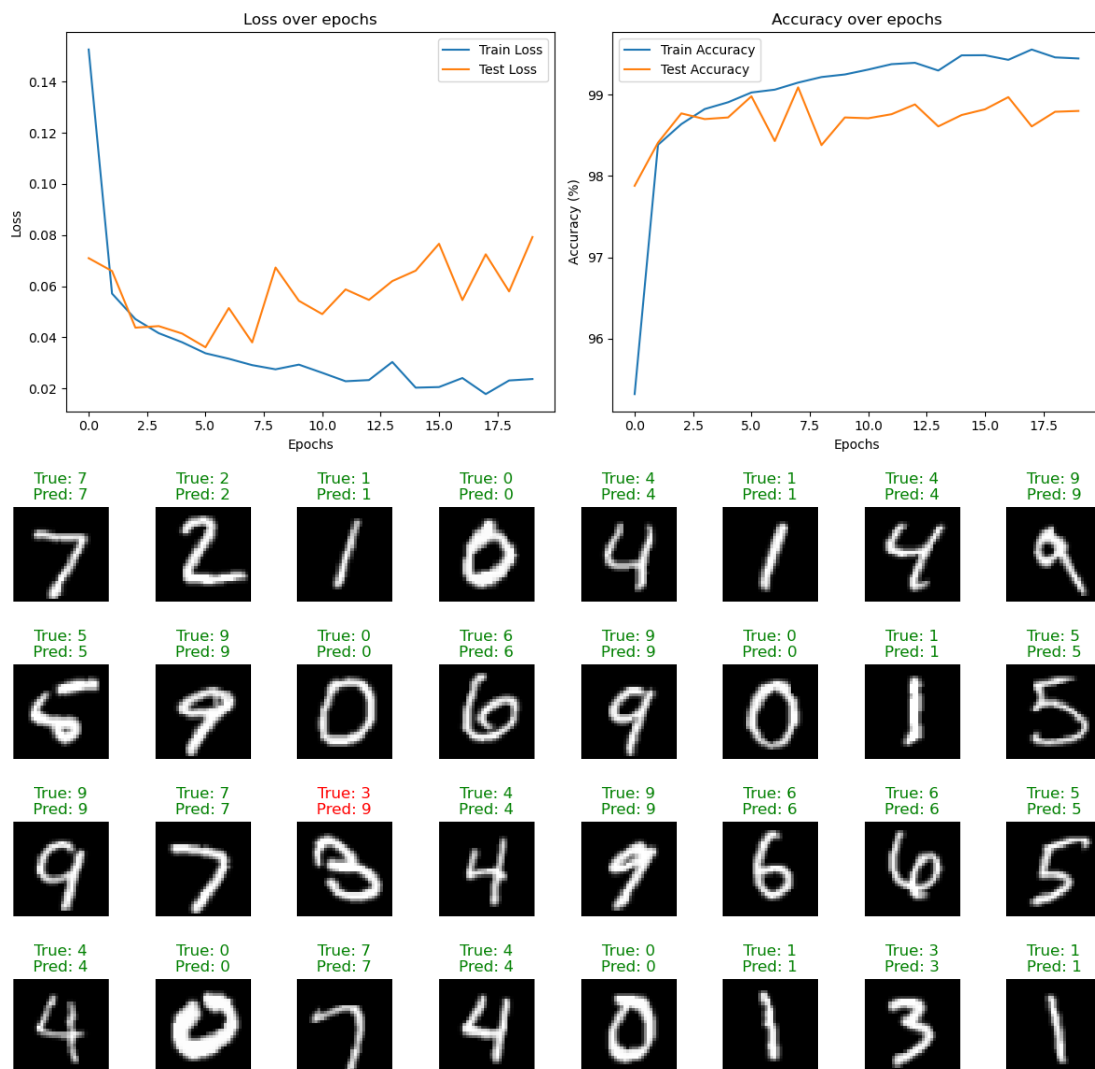
(2) 学习率 $lr=0.005$, 迭代次数 epoch=10 时



Confusion Matrix

	0	1	2	3	4	5	6	7	8	9
0	973	0	2	0	0	0	0	2	1	2
1	0	1130	0	0	0	0	1	3	1	0
2	0	0	1020	0	1	0	0	10	1	0
3	0	0	0	1005	0	3	0	1	1	0
4	0	0	0	0	973	1	0	4	0	4
5	1	0	0	6	0	879	1	1	2	2
6	9	3	1	0	1	4	936	0	4	0
7	0	2	3	0	0	0	0	1020	1	2
8	2	1	2	3	0	2	0	1	959	4
9	0	0	0	6	4	4	1	6	1	987
	0	1	2	3	4	5	6	7	8	9

(3) 学习率 $lr=0.005$, 迭代次数 epoch=20 时



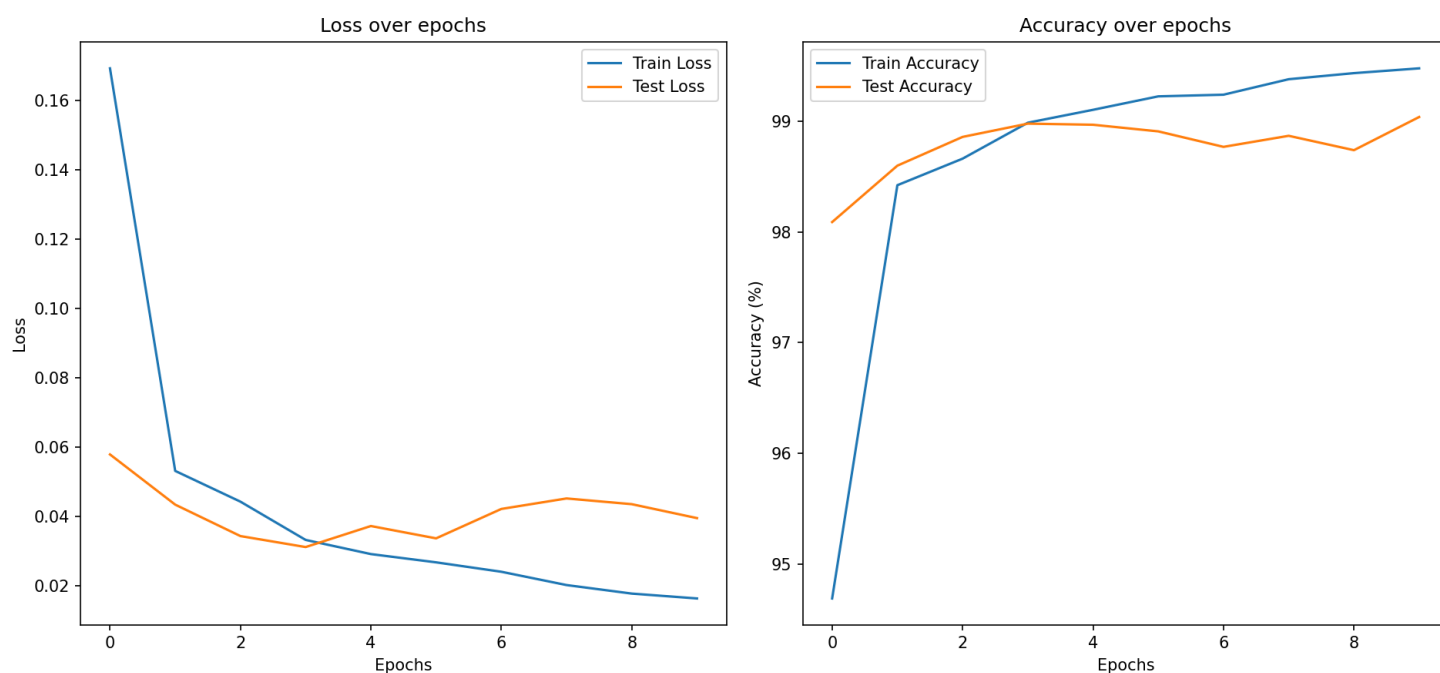
可以看到，学习率固定为 0.005 时，训练过程在前 10 个 epoch 就已经快速收敛：训练损失持续下降、训练准确率稳步上升，到 10 个 epoch 时训练集上的表现已经很高。对应的测试集在前 10 个 epoch 上也有明显提升，但波动比训练集大，测试损失并没有像训练损失那样平滑下降，测试准确率虽高但起伏较小，说明模型在学习到有用特征的同时对测试集的泛化还有一定不稳定性。把训练延长到 20 个 epoch 后，训练损失继续下降、训练准确率接近饱和，而测试损失的下降幅度明显变小甚至出现回升，测试准确率的提升也非常有限，训练-测试之间的差距有所扩

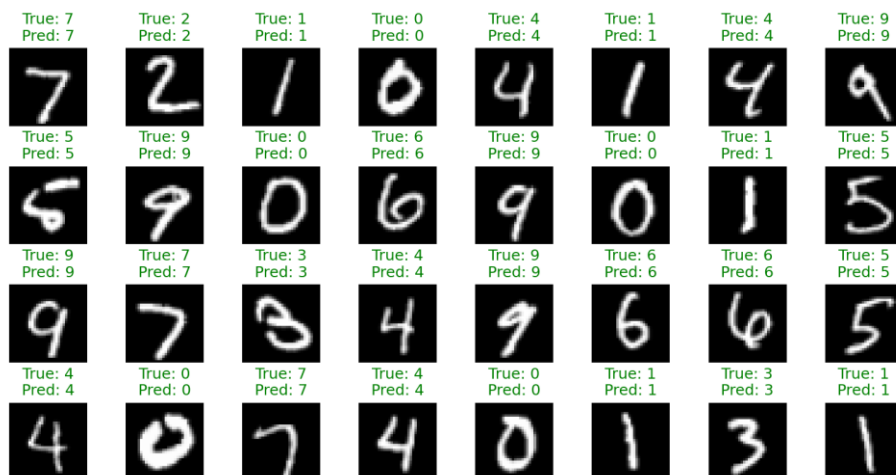
大。直观上这就是典型的“训练集拟合更好、测试集收益递减并出现轻微**过拟合**”的现象：更多的训练轮次把模型参数往训练集误差更低的方向推，但并没有带来等比例的测试集性能提升，反而可能把噪声或训练集特有的细节也学进去了。

这是因为学习率 0.005 对于这个网络和数据来说使得模型收敛较快，短期内能迅速降低训练误差，但较大的步长也会让参数在更深训练阶段在测试误差表面上震荡或**越过最优点**。并且模型容量（LeNet-5 的参数量）相对于 MNIST 来说足够大，随着 epoch 增加容易把**训练集的细微差异**记住，从而出现训练误差持续下降但泛化收益停滞的情况。混淆矩阵和样例图也支持这个结论：大多数样本被正确分类，少数错误集中在形状相近的数字（例如 3/9、4/9、0/6 等），这些错误在更多 epoch 后并没有被显著纠正，说明模型在区分这些边界样本上仍受限于训练数据和模型泛化能力。

（4）学习率 $lr=0.003$ ，迭代次数 epoch=10 时

于是我们减小学习率到 0.003，再看模型的训练效果。





Confusion Matrix

True Label \ Predicted Label	0	1	2	3	4	5	6	7	8	9
0	974	1	1	1	0	0	2	1	0	0
1	0	1131	2	1	0	0	1	0	0	0
2	0	0	1026	0	4	0	0	1	1	0
3	0	0	1	1007	0	2	0	0	0	0
4	0	1	0	0	970	0	0	0	0	11
5	1	0	0	5	0	882	2	0	1	1
6	2	3	0	0	3	1	948	0	1	0
7	0	3	12	3	1	0	0	1009	0	0
8	0	0	2	2	0	2	0	0	968	0
9	0	1	0	3	5	5	0	4	2	989

可以看到，在适当降低学习率后， $lr=0.003$ 、 $epoch=10$ 下的结果显示模型仍然能很快学到有效特征：训练损失稳步下降、训练准确率接近或超过 99%，混淆矩阵对角线占优，绝大多数样本被正确分类，错误主要集中在形状相近的类别（如 3/9、4/9、0/6 等）。测试损失和测试准确率波动较小、曲线更平滑，说明在前 10 个 epoch 内模型的泛化性能比较稳定，未出现明显的训练—测试差距扩大。

把这个结果和之前 $lr=0.005$ （同样 $epoch=10$ ）相比， $lr=0.005$ 时训练损失下降更快、训练准确率上升更迅猛，但测试损失波动更大、后期更容易出现测试性能停滞或轻微回升的迹象；而 $lr=0.003$ 则更新步长更小，参数更新更平滑，训练曲线和测试曲线之间的差距更小，早期过拟合的迹象被抑制了。

4.2 VCG 模型实现手写数字识别

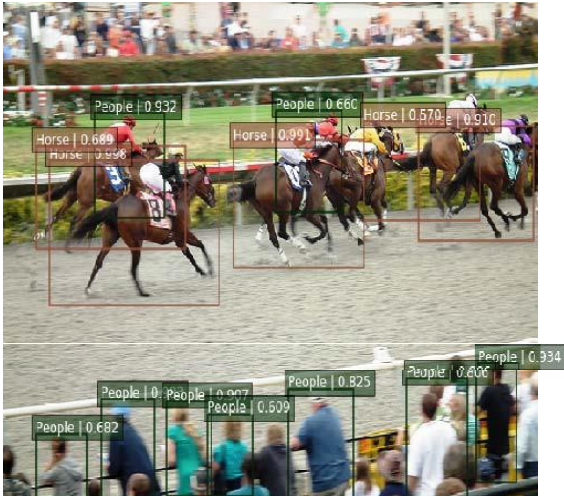
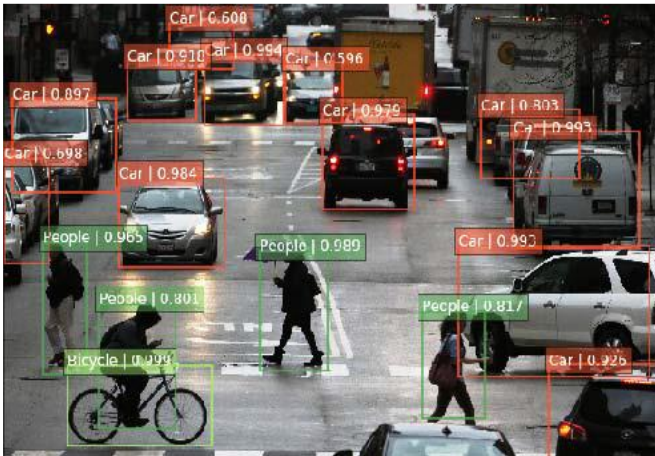
查阅资料后得知，除了 LeNet-5 模型，还有一种 VGG（Visual Geometry Group）模型，是由牛津大学视觉几何组提出的深度卷积神经网络架构。

VGG-16 模型架构包含 13 个卷积层、2 个全连接层以及 1 个 Softmax 分类器。KarenSimonyan 和 AndrewZisserman 在 2014 年的论文《Very Deep Convolutional Network for Large Scale Image Recognition》中提出该模型，构建了由卷积层与全连接层组成的 16 层网络，仅依靠堆叠的 3×3 卷积层。

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGG 作为经典的深度卷积神经网络，在众多领域都有着广泛应用。在图像分类任务里，其预训练模型能在不同数据集上微调，像对 CIFAR-10、ImageNet 等数据集进行分类，精准判断图像中物体类别。目标检测中，它常作为特征提取器，与 RPN 等模块结合，助力行人、车辆检测等任务，完成对目标物体的定位与识别^[4]。

语义分割方面，借助 VGG 提取的特征，可实现对图像像素级分类，区分天空、草地、建筑物等不同区域的像素。在风格迁移里，通过 VGG 不同层次卷积层分别提取内容与风格特征，将一种风格应用到另一种内容图像上。此外，在医学影像分析、人脸识别、OCR 识别，甚至跨模态任务（如图文检索）中，VGG 都凭借其强大的特征提取能力，发挥着重要作用，不过因其计算复杂度高，在实际应用中也面临一些挑战。



针对 MNIST 数据集（ 28×28 像素的灰度图像），本实验设计了一个经过合理简化的 VGG 架构。

因为原始 VGG 网络处理对象的尺寸较大，主要是 224×224 像素的 ImageNet 数据集。而 MNIST 数据集尺寸较小，原始 VGG 架构直接应用于 28×28 的 MNIST 图像可能会使部分空间信息过早丢失无法处理，从而导致训练效果减弱。所以，我通过减小滤波器尺寸、减少卷积层层数、使用正则化等方法提高 VGG 与 MNIST 数据集的适配度，提高模型效果。

(1) 卷积块设计：

每个卷积块由两个卷积层、BN（批量归一化）层、一个池化层、Dropout 层构成，其中卷积块 1、2、3 各自的通道数分别为 64、128、256。

下面展示以卷积块 1 为例的代码。

```
# Block 1
model.add(layers.Conv2D(64, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.Conv2D(64, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2), strides=2))
model.add(layers.Dropout(0.25))
```

(2) 分类层设计

分类层由 Flatten 层、Dense 全连接层、BN（批量归一化）层、Relu 激活构成。

其中在全连接层引入正则化 L2 方法，防止出现过拟合现象。

```
# 分类层
model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu',
kernel_regularizer=regularizers.l2(0.001)))
model.add(layers.BatchNormalization())
model.add(layers.Dropout(0.5))
model.add(layers.Dense(256, activation='relu',
kernel_regularizer=regularizers.l2(0.001)))
model.add(layers.BatchNormalization())
model.add(layers.Dropout(0.5))
model.add(layers.Dense(num_classes, activation='softmax'))
```

关键代码如下所示

```
def build_vgg_mnist(input_shape=(28, 28, 1), num_classes=10):
    model = models.Sequential()

    # 输入适配层（处理小尺寸图像）
    model.add(layers.Conv2D(32, (3, 3), padding='same',
input_shape=input_shape))
    model.add(layers.BatchNormalization())
    model.add(layers.Activation('relu'))

    # Block 1
    model.add(layers.Conv2D(64, (3, 3), padding='same'))
    model.add(layers.BatchNormalization())
```

```

model.add(layers.Activation('relu'))
model.add(layers.Conv2D(64, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2), strides=2))
model.add(layers.Dropout(0.25))

# Block 2
model.add(layers.Conv2D(128, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.Conv2D(128, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2), strides=2))
model.add(layers.Dropout(0.25))

# Block 3
model.add(layers.Conv2D(256, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.Conv2D(256, (3, 3), padding='same'))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2), strides=2))
model.add(layers.Dropout(0.25))

# 分类层
model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu',
kernel_regularizer=regularizers.l2(0.001)))
model.add(layers.BatchNormalization())
model.add(layers.Dropout(0.5))
model.add(layers.Dense(256, activation='relu',
kernel_regularizer=regularizers.l2(0.001)))
model.add(layers.BatchNormalization())
model.add(layers.Dropout(0.5))
model.add(layers.Dense(num_classes, activation='softmax'))

return model

# 构建 VGG 模型
vgg_model = build_vgg_mnist()
vgg_model.summary()

```

```

# 编译模型
vgg_model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001)
,
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

# 训练配置
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=8, restore_best_weights=True)
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss', factor=0.2, patience=4, min_lr=1e-7)

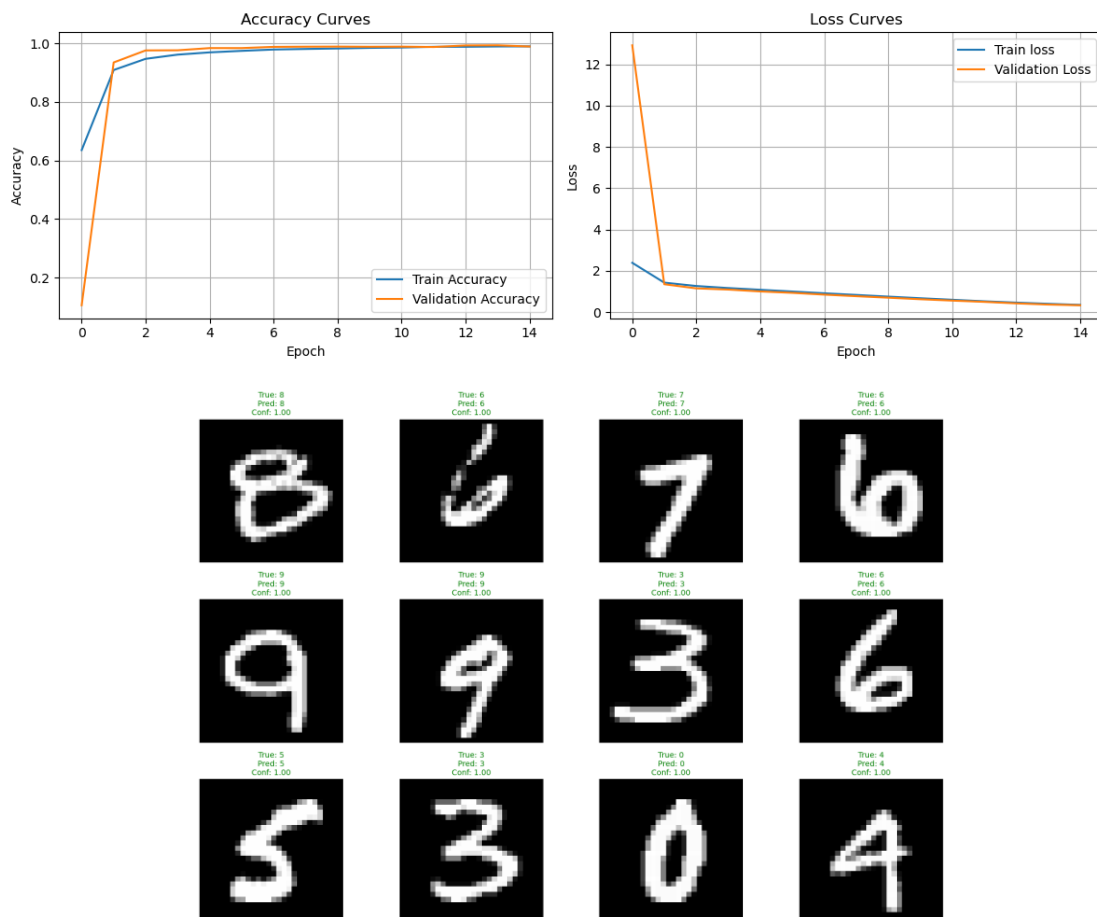
# 训练模型
history_vgg = vgg_model.fit(
    train_images, train_labels,
    epochs=30,
    batch_size=128,
    validation_split=0.2,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

# 评估模型
test_loss, test_acc = vgg_model.evaluate(test_images, test_labels)
print(f'\nVGG 测试准确率: {test_acc:.4f}')

```

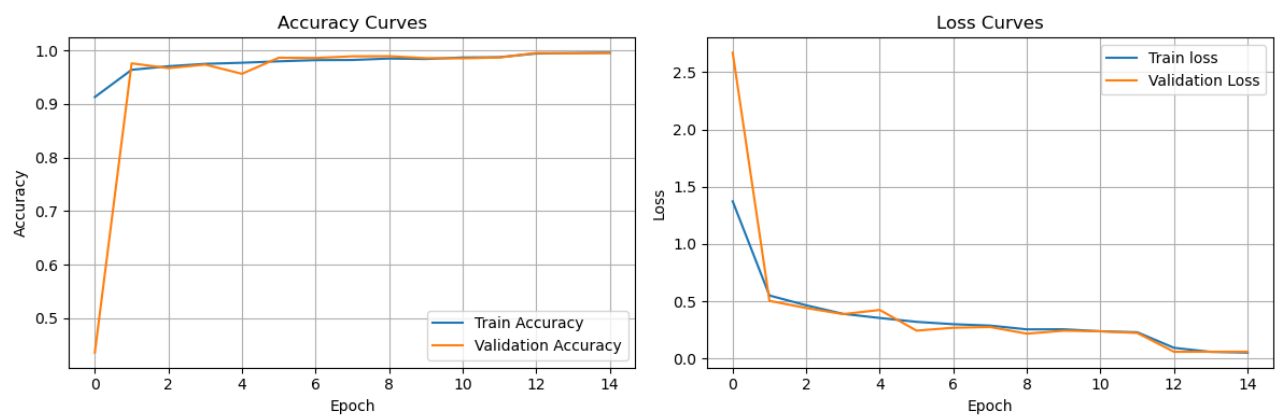
先用一层小卷积核做输入适配并接 BatchNorm+ReLU，然后通过三组卷积块逐步提取特征，每组包含多次卷积、BatchNorm、ReLU，随后用最大池化缩小空间维度并用 Dropout 降低过拟合风险；特征提取后展平并接两层带 L2 正则化的全连接层（每层后也有 BatchNorm 和较大比例的 Dropout），最后用 softmax 输出 10 类概率。模型用 Adam 优化器和稀疏交叉熵编译，训练时通过 EarlyStopping 提前停止并恢复最佳权重，同时用 ReduceLROnPlateau 在验证损失停滞时降低学习率，训练结束后在测试集上评估并打印准确率。

(1) 学习率=0.001, 迭代次数=15



验证结果：测试准确率：0.9908，测试损失：0.3256，模型的准确率较高，损失率有一定下降空间，模型效果较好。

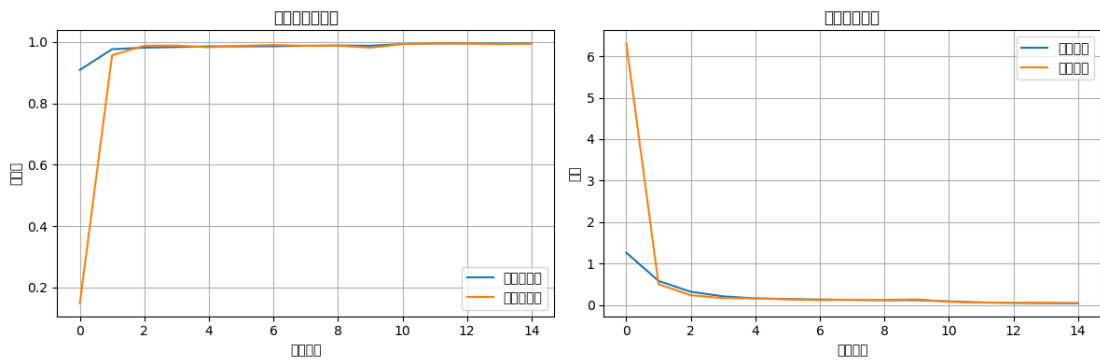
(2) 学习率=0.0001, 迭代次数=15





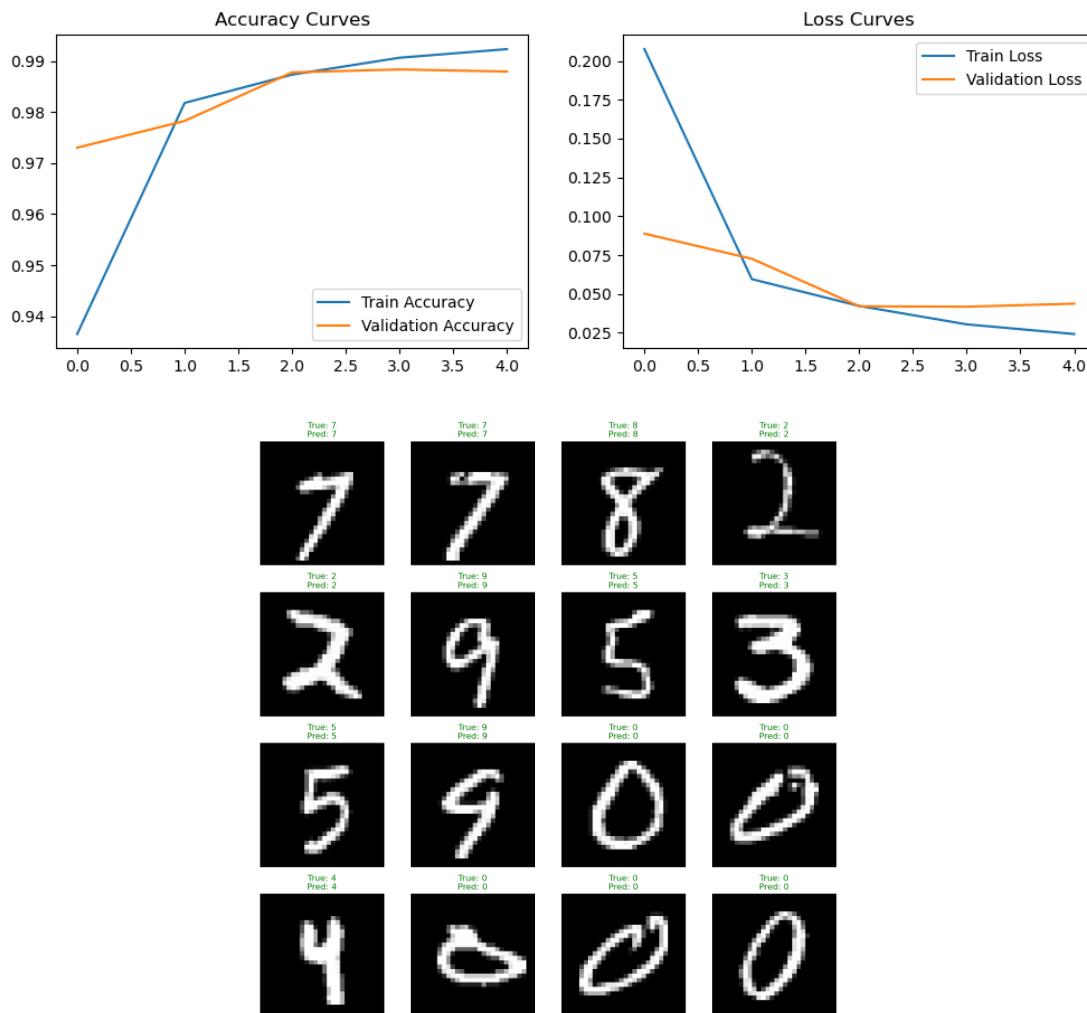
验证结果：测试准确率：0.9942，测试损失：0.0583，准确率有一定提升，随着迭代次数增加，准确率曲线和损失率整体呈现缓慢变化趋势，但出现了局部的抖动，识别效果展示中出现了错误。

(3) 学习率=0.001，迭代次数=10



测试准确率：0.9951，测试损失：0.0399，准确率有一定提升，随着迭代次数增加，准确率曲线和损失率曲线基本稳定。

(4) 学习率=0.0001，迭代次数=30



测试准确率为 0.9872，损失率为 0.0504，当迭代轮数达到 30，学习率为 0.0001，曲线收敛速度变慢，在初期效果不佳，可能是由于学习率较低导致；且测试损失率随着迭代次数增加，损失率上升，说明出现了过拟合问题，训练次数的过多导致模型对训练集中的噪声提取增强，模型的泛化能力下降，应当适当调整迭代次数。

结论

通过上述实验过程我们可以得到，VGG 对 MNIST 数据集的训练准确度基本超过 99%。在学习率=0.01、迭代次数=10 的组合里，得到了较好的结果，证明模型已经

具备了完善的预测机制，能够适配 MNIST 数据集，处理类似的识别任务，但在迭代次数较高的情况下仍出现了过拟合情况，

5. 参考文献

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Hinton, G. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- [2] Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, 25, 1097-1105.
- [3] Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 740-748.
- [4] Redmon, J., Farhadi, A., & Zisserman, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 779-788.
- [5] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 448-456.
- [6] Karpathy, A., Vinyals, O., Le, Q. V., & Bengio, Y. (2015). Multimodal Neural Architectures for Visual Question Answering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2821-2829.
- [7] Donahue, J., Vinyals, O., Kavukcuoglu, K., & Le, Q. V. (2015). Long-term Recurrent Convolutional Networks for Visual Question Answering. *Proceedings of the IEEE*

Conference on Computer Vision and Pattern Recognition, 2830-2838.

- [8] Vinyals, O., Erhan, D., Le, Q. V., & Bengio, Y. (2015). Show and Tell: A Neural Image Caption Generator. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2812-2820.
- [9] Razavian, A., & Ullman, S. (2014). Deep Convolutional Features for Text Classification. Proceedings of the Conference on Empirical Methods in Natural Language Processing, 1538-1547.
- [10] Donahue, J., Vinyals, O., Kavukcuoglu, K., & Le, Q. V. (2015). Long-term Recurrent Convolutional Networks for Visual Question Answering. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2830-2838.
- [11] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 448-456.
- [12] Donahue, J., Vinyals, O., Kavukcuoglu, K., & Le, Q. V. (2015). Long-term Recurrent Convolutional Networks for Visual Question Answering. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2830-2838.
- [13] 岳斌.基于脉冲神经网络的手写数字识别系统的设计与实现[D].中国科学院大学 (中国科学院深圳先进技术研究院),2022.DOI:10.27822/d.cnki.gszxj.2022.000152.
- [14] Su, H., Wang, Z., Zhang, H., & Li, L. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 448-456.
- [15] Redmon, J., Farhadi, A., & Zisserman, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 779-788.

- [16] Lin, T., Deng, J., ImageNet, & Krizhevsky, A. (2014). Microsoft COCO: Common Objects in Context. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 740-748.
- [17] Razavian, A., & Ullman, S. (2014). Deep Convolutional Features for Text Classification. Proceedings of the Conference on Empirical Methods in Natural Language Processing, 1538-1547.
- [18] Vinyals, O., Erhan, D., Le, Q. V., & Bengio, Y. (2015). Show and Tell: A Neural Image Caption Generator. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition,
- [19] Choudhary, A.; Ahlawat, S.; Rishi, R. A binarization feature extraction approach to OCR: MLP vs. RBF. In Proceedings of the International Conference on Distributed Computing and Technology ICDCIT, Bhubaneswar, India, 6–9 February 2014; Springer: Cham, Switzerland, 2014; pp. 341–346
- [20] Ciresan, D.C.; Meier, U.; Masci, J.; Gambardella, L.M.; Schmidhuber, J. High-performance neural networks for visual object classification. *arXiv* **2011**, arXiv:1102.0183v1
- [21] Alvear-Sandoval, R.; Figueiras-Vidal, A. On building ensembles of stacked denoising auto-encoding classifiers and their further improvement. *Inf. Fusion* **2018**, *39*, 41–52.
- [22] Xie, Z.; Sun, Z.; Jin, L.; Feng, Z.; Zhang, S. Fully convolutional recurrent network for handwritten chinese text recognition. In Proceedings of the 23rd International Conference on Pattern Recognition (ICPR 2016), Cancun, Mexico, 4–8 December 2016.
- [23] Wu, Y.-C.; Yin, F.; Liu, C.-L. Improving handwritten chinese text recognition using neural network language models and convolutional neural network shape

models. *Pattern Recognit.* **2017**, 65, 251–264.

附件：设计程序

北京理工大学

仿真设计报告

报告题目 雷达目标识别仿真设计报告

学 号 1120230621

姓 名 闫子易

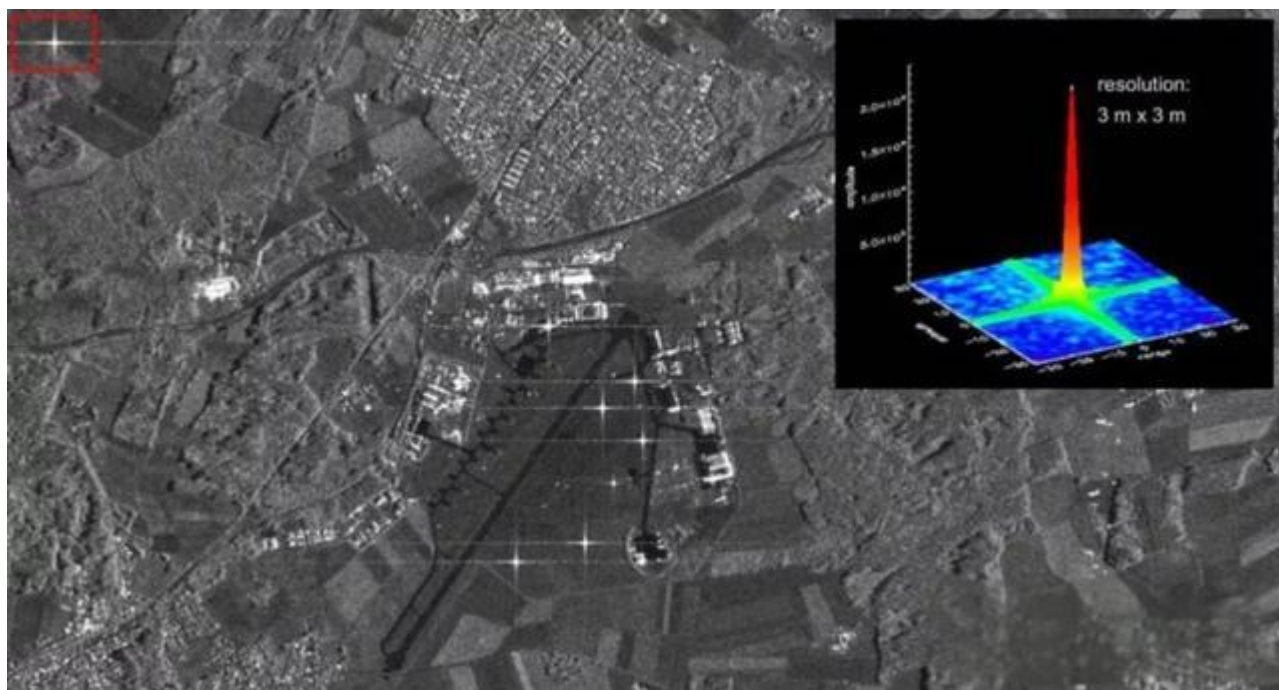
学 科 电子科学与技术

学 院 集成电路与电子学院

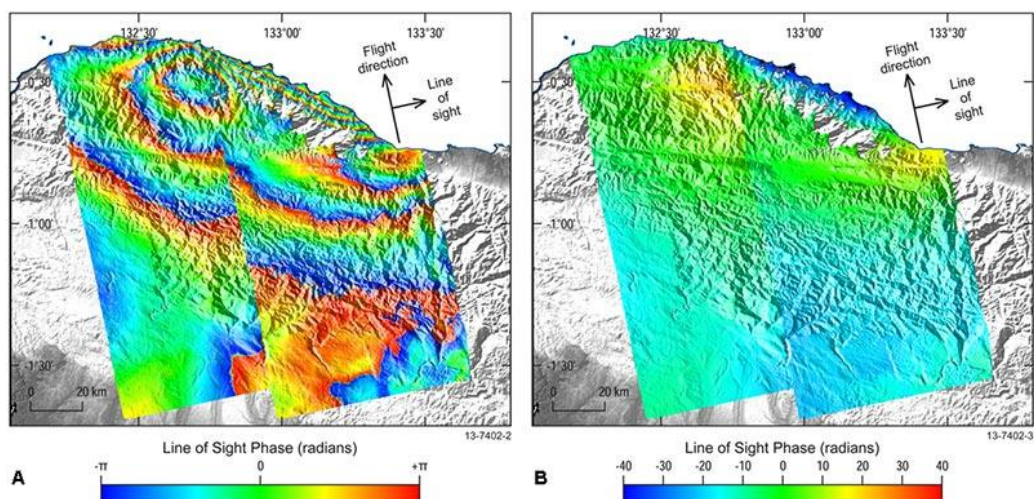
2025 年 11 月 30 日

1. 合成孔径雷达目标识别技术发展历程

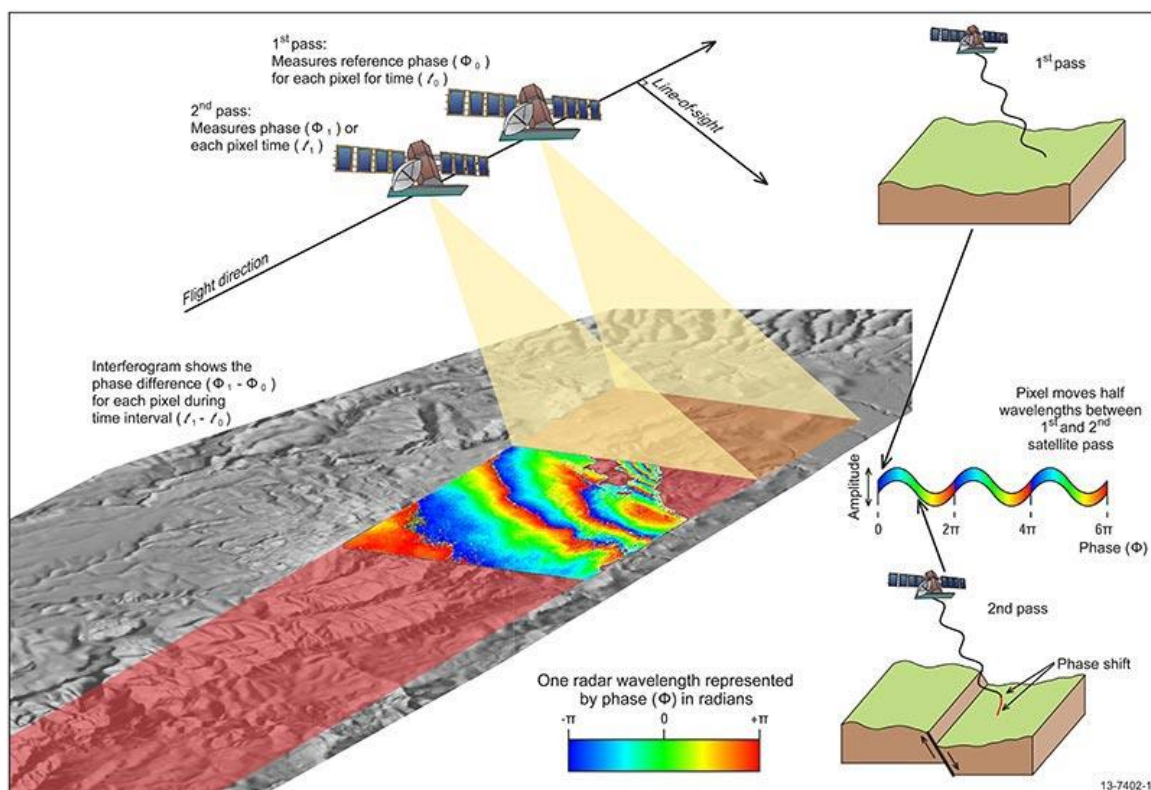
合成孔径雷达（Synthetic Aperture Radar，SAR）是上世纪 50 年代提出并研制成功的一种微波遥感设备，通过在平台上搭载天线来发射和接收电磁波来探测地面目标对电磁波的调制特性，属于主动遥感方式^[1]。在对地观测时，能克服云、雾、雨、雪等恶劣天气的干扰以及黑暗条件的影响，其全天时、全天候、强穿透性、高精度、低成本和连续空间覆盖的特点，使其在国民经济诸多领域具有独特的优势，同时在农业、林业、地质和水文等方面具有广泛的应用前景，近年来受到世界各国高度重视并得到迅速发展^[2]。



SAR 设备在目标场景被照射期间移动的距离会形成较大的合成天线孔径（即天线尺寸）^[2]。通常情况下，孔径越大，图像分辨率越高，无论孔径是物理的（大型天线）还是合成的（移动天线）——这使得 SAR 能够使用相对较小的物理天线生成高分辨率图像。对于固定尺寸和方向的天线，距离较远的物体会被照亮更长时间——因此，SAR 具有为更远的物体创建更大合成孔径的特性，从而在一定范围的观测距离内实现一致的空间分辨率^[3]。



为了生成合成孔径雷达（SAR）图像，需要连续发射无线电波脉冲来“照射”目标场景，并接收和记录每个脉冲的回波。发射脉冲和接收回波均使用单个波束成形天线，波长范围从米级到几毫米级^[4]。随着飞机或航天器上的 SAR 设备移动，天线相对于目标的位置也会随时间变化。对连续记录的雷达回波进行信号处理，可以将来自多个天线位置的记录进行组合^[5]。这一过程形成合成天线孔径，从而生成比使用特定物理天线所能达到的更高分辨率的图像。



1978 年, 美国宇航局发射了世界上第一颗 SAR 卫星 Seasat-A, 这标志着合成孔径雷达进入了对地观测的时代。在此之后, 美国宇航局利用航天飞机分别于 1981 年、1984 年和 1994 年将 SIR-A、SIR-B、SIR-C 成像雷达送入太空^[6]。合成孔径雷达由最初的 HH 单极化^[7]、L 波段 SAR 卫星, 逐渐发展到具有 4 种极化方式 (HH、HV、VV、VH)、多波段 (L、C、X) 的雷达系统。

1989 年, 加拿大航天局开始进行 SAR 卫星—RADARSAT-1 的研制, 并于 1995 年, 1996 年正式工作, 是加拿大的第 1 颗商业对地观测卫星, 主要用于地球环境和自然资源监测。该卫星运行在近极地太阳同步轨道上, 工作方式为 C 波段、HH 极化, 首次采用了可变视角的 ScanSAR 工作模式。RADARSAT-2 是继 RADARSAT-1 之后的新一代商用合成孔径雷达卫星, 它在原有的基础上增加了多极化成像, 3m 分辨率成像、双边 (dual-channel) 成像和动目标探测 (MODEX)。

1991 年和 1995 年, 欧空局分别发射了欧洲遥感卫星系列民用雷达成像卫星: ERS-1 和 ERS-2^[8], 主要用于对陆地、海洋、冰川、海岸线等成像, 该系列卫星采用了 C 波段、VV 极化的工作方式, 可以获得 30m 空间分辨率和 100km 观测带宽的高质量图像。

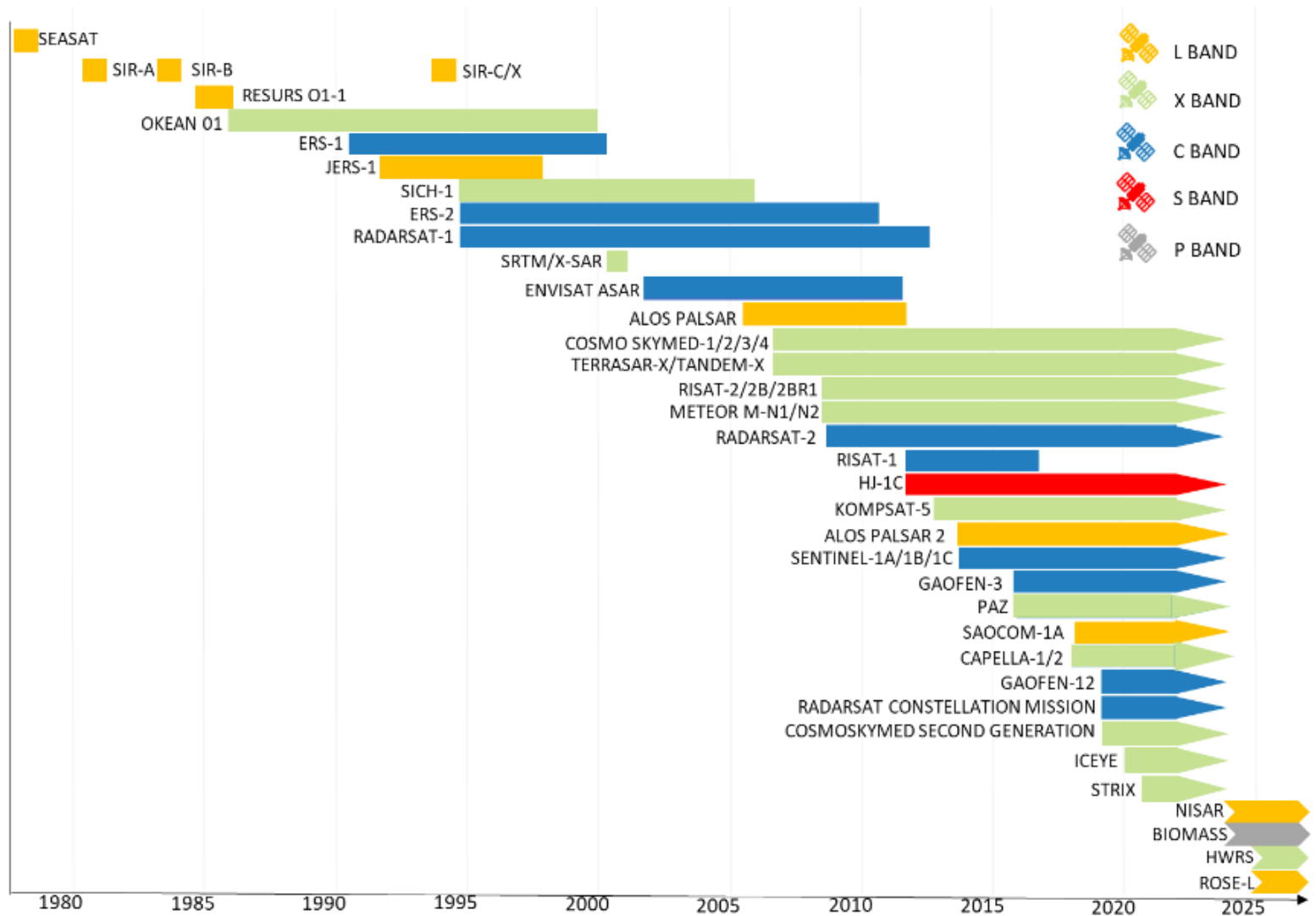
1992 年, 日本发射了 JERS-1 卫星^[9], 该卫星运行在近极地太阳同步轨道上, 其入射角固定、HH 极化, 工作在 L 波段, 分辨率为 18m。主要用于地质研究、农林业、海洋观测、地理测绘、环境灾害监测等。

2002 年, 欧空局发射了近太阳同步轨道雷达成像卫星 Envisat, 该卫具有多种极化、可变入射角、大幅宽等特点, 主要用于对地观测和地球环境的研究。

2007 年, 意大利国防部与航天局合作发射了雷达成像卫星 Cosmo-Skymed, Cosmo-Skymed 卫星为 X 波段, 具有多极化、多入射角的特性, 具备 3 种工作方式和 5 种分辨率的成像模式: ScanSAR (100m 和 30m)、Strip-Map (3m 和 1.5m)、

SpotLight (1m)。作为全球第一个分辨率高达 1m 的雷达成像卫星星座，Cosmo-Skymed 系统主要用于环境资源监测、灾害监测、海事管理等方面^[10]。

同年，德国宇航中心（DLR）和民营企业 EADS Astrium 及 Infoterra 公司共同发射了军民两用侦察卫星 TerraSAR-X。该卫星在近极地太阳同步轨道上运行，采用了 X 波段、多极化的方式，具备 4 种工作方式和 4 种不同分辨率的成像模式：StripMap（单视情况：3m，3m）、Scan-SAR（4 视情况：15m，16m）、Spot-Light（单视情况：2m，1.2m）和高分辨 SpotLight（单视情况：1m，1.2m）。



2012 年，中国发射了首颗民用 SAR 卫星“环境一号 C 星（HJ-1C）”，该卫星工作在 S 波段，具备 5 米分辨率（条带模式）和 20 米分辨率（扫描模式），主要用于环境监测和地质灾害预报。

2016 年 8 月 10 号，高分三号卫星发射升空，这是我国首颗分辨率达到 1 米的 C 波段 SAR 卫星，该卫星具有高分辨率、大成像幅宽、高定量精度、多成像模式、长时工作和长寿命运行等特点，整星综合技术水平超过国际同类卫星。

在星载合成孔径雷达这条道路上，我们是后来者，但绝不落后，我们在不断前进，不断进步，期待具有中国特色、高精度的星载合成孔径雷达系统的成功应用。



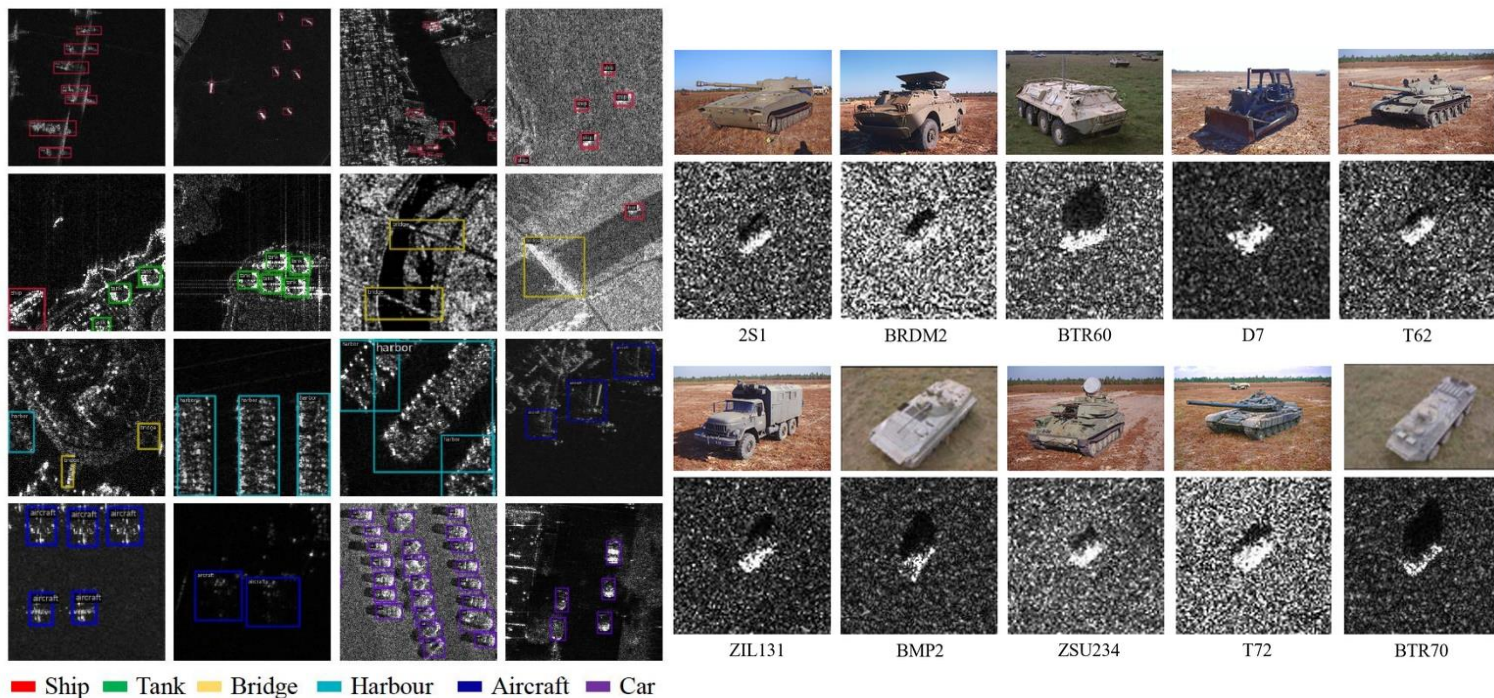
2. Moving and Stationary Target Acquisition and Recognition(MSTAR)数据集简介

MSTAR (The Moving and Stationary Target Acquisition and Recognition)^[11] 数据集由美国桑迪亚国家实验室 (Sandia national laboratory) 在二十世纪 90 年代收集并发布。MSTAR 数据集包含多种目标及变体、不同的场景和观测条件、旋转角与擦地角变换^[12]。由 MSTAR 数据集可构建标准工作条件 (Standard Operating Condition, SOC) 和扩展工作条件 (Extended Operating Conditon, EOC) 数据集。即目标、环境和成像条件差异细微和剧烈两种情况。

目前, MSTAR 数据集已在 SAR 地面目标识别领域中广泛应用。而近年来, SOC_s 和 EOC_s 的构建大多基于复旦大学徐丰教授团队在 A-ConvNets 一文中所设置的实验, 如擦地角变换、目标版本配置不同以及信噪比变化。需要指出的是, 一是现有文献并未充分利用 MSTAR 数据集中所有的公开数据; 二是 SAR 自动目标识别中需要考虑的 EOC_s 情况远不止上述情况, 包括目标、环境和传感器等因素[4]。文献[11]详细介绍了近年来基于 MSTAR 数据集的实验设置和方法发展, 可以发现 MSTAR 数据集已逐渐难以满足如今研究需求。

采集该数据集的传感器为高分辨率的聚束式合成孔径雷达, 该雷达的分辨率为 $0.3\text{m} \times 0.3\text{m}$ ^[13]。工作在 X 波段, 所用的极化方式为 HH 极化方式。对采集到的数据进行前期处理, 从中提取出像素大小为 128×128 包含各类目标的切片图像。该数据大多是静止车辆的 SAR 切片图像, 包含多种车辆目标在各个方位角下获取到的目标图像^[14]。

MSTAR 混合目标数据中包含十类军事目标的切片图像, 这些军事目标分别为 2S1 (自行榴弹炮)、BRDM2 (装甲侦察车)、BTR60 (装甲运输车)、D7 (推土机)、T62 (坦克)、ZIL131 (货运卡车)、ZSU234 (自行高炮)、T72。这些目标是雷达工作在多种不同的俯仰角时, 各个目标在方向上面的成像图片^[15]。



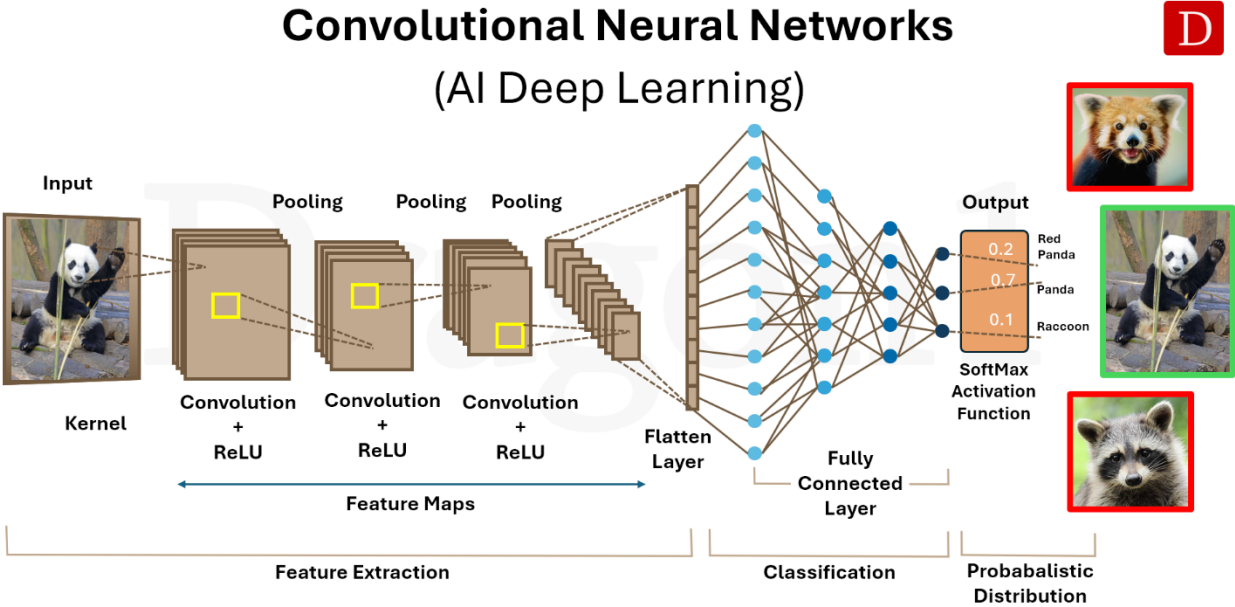
数据分为标准工作条件（SOC）和扩展工作条件（EOC），标准工作条件是测试与训练 SAR 图像目标外形配置和型号相同，仅成像时目标的俯仰角和方位角不同，扩展工作条件是指测试与训练 SAR 图像有很大不同，主要是成像角度的改变、外形配置的变化以及型号不同^[16]。

除了 10 类军事目标外，MSTAR 数据集还提供了大幅场景 SAR 图像，包含森林、地面、建筑等杂波，可用于目标检测和识别。

3. 卷积神经网络雷达目标识别原理

卷积神经网络（CNN）是一类前馈神经网络，通过优化滤波器（或卷积核）来自动学习特征。这种深度学习网络已被广泛用于处理并预测多种类型的数据，包括文本、图像和音频。CNN 已成为基于深度学习的计算机视觉和图像处理方法的事实标准，直到近来在某些场景下才被诸如 Transformer 之类的新型架构部分取代^[20]。

早期神经网络在反向传播时常遇到梯度消失或梯度爆炸的问题，而卷积网络通过共享权重并减少连接数带来的正则化效应，有助于缓解这些问题。举例来说，要处理一张 100×100 像素的图像，若使用全连接层，每个神经元可能需要约 10,000 个权重；而采用级联的卷积（或互相关）核来处理 5×5 的图块时，每层卷积只需约 25 个权重。与低层特征相比，高层特征能从更宽的上下文窗口中提取信息。



© Copyright 2025, Dragon1, www.dragon1.com

图中所示为该卷积神经网络（CNN）的结构。整个网络可分为两部分。左侧由四个卷积模块组成，每个模块包含四层：卷积层、批量归一化、线性整流单元（ReLU）和最大池化层。输入图像依次通过这四个模块后，可提取出富含视觉信息的特征图；随后将这些特征图展平为一个维度为 2048 的特征向量，该向量作为 CNN 非线性分类器的输入。分类器部分由三层全连接层构成，层间使用 ReLU 激活，最后以 softmax 函数输出用于多类预测。

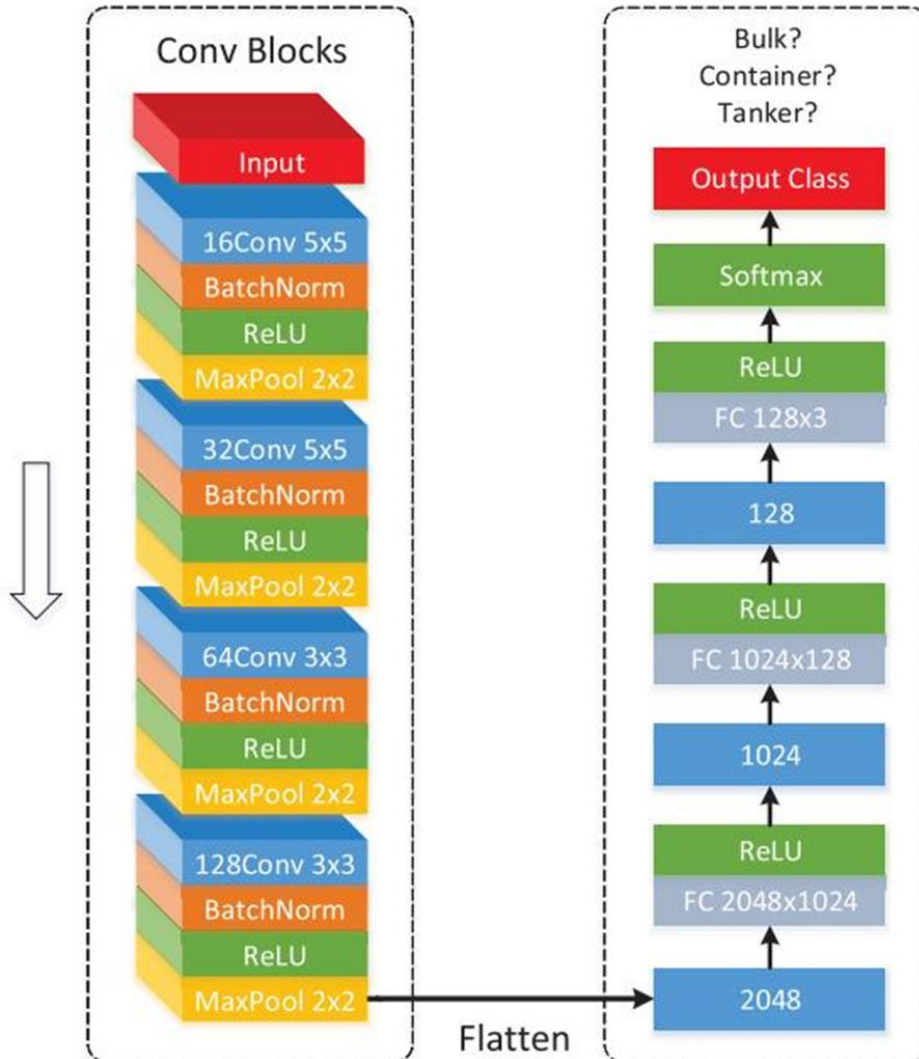
首先，按常规图像分类方式对 CNN 进行训练。设输入图像为 $x_i \in \mathbb{R}^{c \times h \times w}$ ，其中三个维度分别表示通道数、图像高度和图像宽度。本文采用交叉熵损失，总损失函数可表示为：

$$J(\theta) = -\frac{1}{m} \sum_i \sum_{c=1}^N \mathbf{1}(y_i = c) \log P_{\theta}(y_i = c | x_i)$$

我们使用随机梯度下降（SGD）来最小化该损失。其中， m 为所有训练样本数， i 为样本索引， N 为类别总数， c 为类别索引， y_i 为第 i 个样本的真实标签， θ 表示CNN的参数。指示函数 $\mathbf{1}(y_i = c)$ 表示当第 i 个样本的真实标签等于 c 时取值为1，否则为0。条件概率 $P_{\theta}(y_i = c | x_i)$ 在第三个全连接层（该层后接ReLU）输出的特征向量上通过softmax计算^[17]，具体为：

$$P_{\theta}(y_i = c | x_i) = \frac{\exp(f_{\theta}(x_i)_c)}{\sum_{k=1}^N \exp(f_{\theta}(x_i)_k)}$$

其中 $f_{\theta}(x_i)_c$ 表示特征向量 $f_{\theta}(x_i)$ 的第 c 个分量。



对于 MSTAR, CNN 的输出维度为 10, 第三个全连接层为 128×10 。训练时, 将训练集中的图像作为 CNN 的输入, 进行前向传播, 得到一个长度为 10 (MSTAR) 的输出向量^[18]。向量的每个分量表示该图像属于某一类别的概率。随后使用随机梯度下降 (SGD) 最小化式 (1)。初始学习率设为 0.001, 每两轮训练将学习率减半, 每五轮进行一次验证。当验证准确率不再上升并出现下降趋势时, 采用早停策略^[19]。

验证阶段, 将验证集的图像输入 CNN, 样本的类别预测由下式给出:

$$\hat{y}_i = \operatorname{argmax}_c P_{\theta}(y_i = c | x_i).$$

测试阶段同样使用测试集图像作为输入, 按上式的方法预测类别标签。



4. 基于 PyTorch 实现的卷积神经网络雷达目标识别设计

4.1 MSTAR CNN 分类

我在这段代码里先用自定义的 logger 打开终端日志并根据是否有 GPU 自动选择设备，然后定义了一个类似 AlexNet 风格的卷积网络：前面用**五个卷积层**和**三个池化层**提取特征，卷积后展平接**两个全连接层**并加 Dropout 做正则化，最后用 LogSoftmax 输出类别概率以配合交叉熵损失。数据部分把所有图像统一缩放到 100×100 ，并把灰度图扩展成三通道以适配网络的输入结构。数据**预处理**把输入统一缩放到指定尺寸、把灰度图扩展为三通道并做标准化，训练和验证数据通过 DataLoader 批量读取；训练循环里每个 epoch 做前向、反向传播和参数更新，同时统计并保存训练/验证的损失与准确率。优化器选用带动量的 SGD，**损失函数**用 CrossEntropyLoss，训练结束后在验证集上收集预测结果用于可视化：绘制损失与准确率曲线、绘制混淆矩阵并展示若干样本的真实与预测标签以便直观检查模型表现，最后停止日志并保存路径。

关键代码如下：

```
class ConvNet(nn.Module):
    def __init__(self, dropout1=0.5, dropout2=0.5, num_classes=10):
        super(ConvNet, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=96,
kernel_size=11, stride=4, padding=5)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=4)
        self.conv2 = nn.Conv2d(96, 256, kernel_size=5, padding=2)
        self.pool2 = nn.MaxPool2d(kernel_size=3, stride=1)
        self.conv3 = nn.Conv2d(256, 384, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(384, 384, kernel_size=3, padding=1)
        self.conv5 = nn.Conv2d(384, 256, kernel_size=3, padding=1)
        self.pool3 = nn.MaxPool2d(kernel_size=3, stride=1)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(1024, 1024)
        self.dropout1 = nn.Dropout(dropout1)
        self.fc2 = nn.Linear(1024, 1024)
        self.dropout2 = nn.Dropout(dropout2)
```



```

        self.fc3 = nn.Linear(1024, num_classes)

    def forward(self, x):
        x = nn.ReLU()(self.conv1(x))
        x = self.pool1(x)
        x = nn.ReLU()(self.conv2(x))
        x = self.pool2(x)
        x = nn.ReLU()(self.conv3(x))
        x = nn.ReLU()(self.conv4(x))
        x = nn.ReLU()(self.conv5(x))
        x = self.pool3(x)
        x = torch.flatten(x, 1)
        x = nn.ReLU()(self.fc1(x))
        x = self.dropout1(x)
        x = nn.ReLU()(self.fc2(x))
        x = self.dropout2(x)
        x = self.fc3(x)
        x = nn.LogSoftmax(dim=1)(x)
        return x

# 增强的训练函数，跟踪指标
def train(model, train_loader, criterion, optimizer, epochs,
validation_loader=None):
    train_losses = []
    train_accs = []
    val_losses = []
    val_accs = []

    for epoch in range(epochs):
        # 训练阶段
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            _, predicted = torch.max(outputs, 1)

```

```

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(train_loader.dataset)
    epoch_acc = correct / total
    train_losses.append(epoch_loss)
    train_accs.append(epoch_acc)

# 验证阶段
val_loss = 0.0
val_correct = 0
val_total = 0
if validation_loader:
    model.eval()
    with torch.no_grad():
        for images, labels in validation_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item() * images.size(0)
            _, predicted = torch.max(outputs, 1)
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

    val_loss = val_loss / len(validation_loader.dataset)
    val_acc = val_correct / val_total
    val_losses.append(val_loss)
    val_accs.append(val_acc)

    print(f"Epoch {epoch+1}/{epochs} | "
          f"Train Loss: {epoch_loss:.4f} | "
          f"Train Acc: {epoch_acc:.4f} | "
          f"Val Loss: {val_loss:.4f} | "
          f"Val Acc: {val_acc:.4f}")
else:
    print(f"Epoch {epoch+1}/{epochs} | "
          f"Train Loss: {epoch_loss:.4f} | "
          f"Train Acc: {epoch_acc:.4f}")

return train_losses, train_accs, val_losses, val_accs

# 可视化函数 - 直接显示图形
def visualize_results(train_losses, train_accs, val_losses, val_accs,
                      y_true, y_pred, class_names, images, true_labels,
                      pred_labels):

```

"""创建并显示所有可视化图形"""

1. 损失和准确率曲线

```
plt.figure(figsize=(14, 5))

# 损失曲线
plt.subplot(1, 2, 1)
plt.plot(train_losses, 'b-', label='Train Loss')
if val_losses:
    plt.plot(val_losses, 'r-', label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# 准确率曲线
plt.subplot(1, 2, 2)
plt.plot(train_accs, 'b-', label='Train Accuracy')
if val_accs:
    plt.plot(val_accs, 'r-', label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show(block=False) # 非阻塞显示
```

2. 混淆矩阵

```
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.show(block=False)
```

3. 样本预测结果

```
plt.figure(figsize=(15, 10))
for i in range(min(24, len(images))): # 显示 24 个样本
    ax = plt.subplot(4, 6, i + 1)
```

```

# 转换为 numpy 数组并反标准化
img = images[i].cpu().permute(1, 2, 0).numpy()
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])
img = std * img + mean
img = np.clip(img, 0, 1)

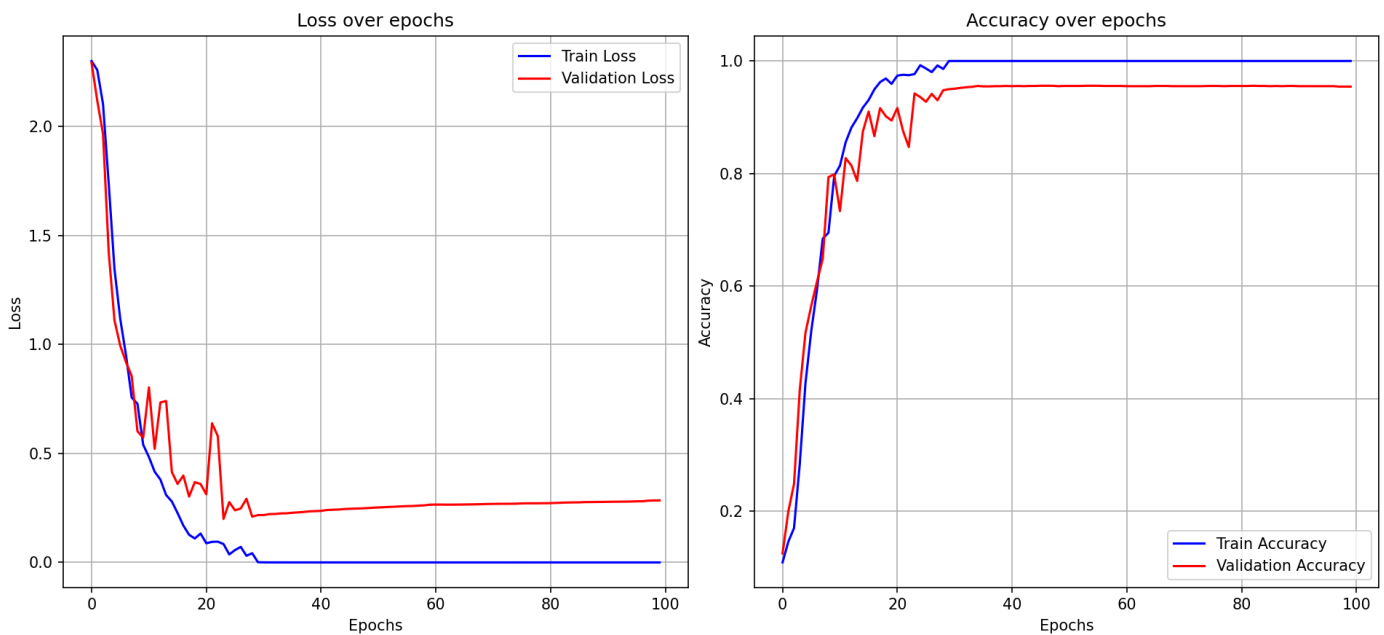
plt.imshow(img)
true_class = class_names[true_labels[i]]
pred_class = class_names[pred_labels[i]]
color = 'green' if true_labels[i] == pred_labels[i] else 'red'
plt.title(f"True: {true_class}\nPred: {pred_class}", color=color,
fontsize=10)
plt.axis('off')

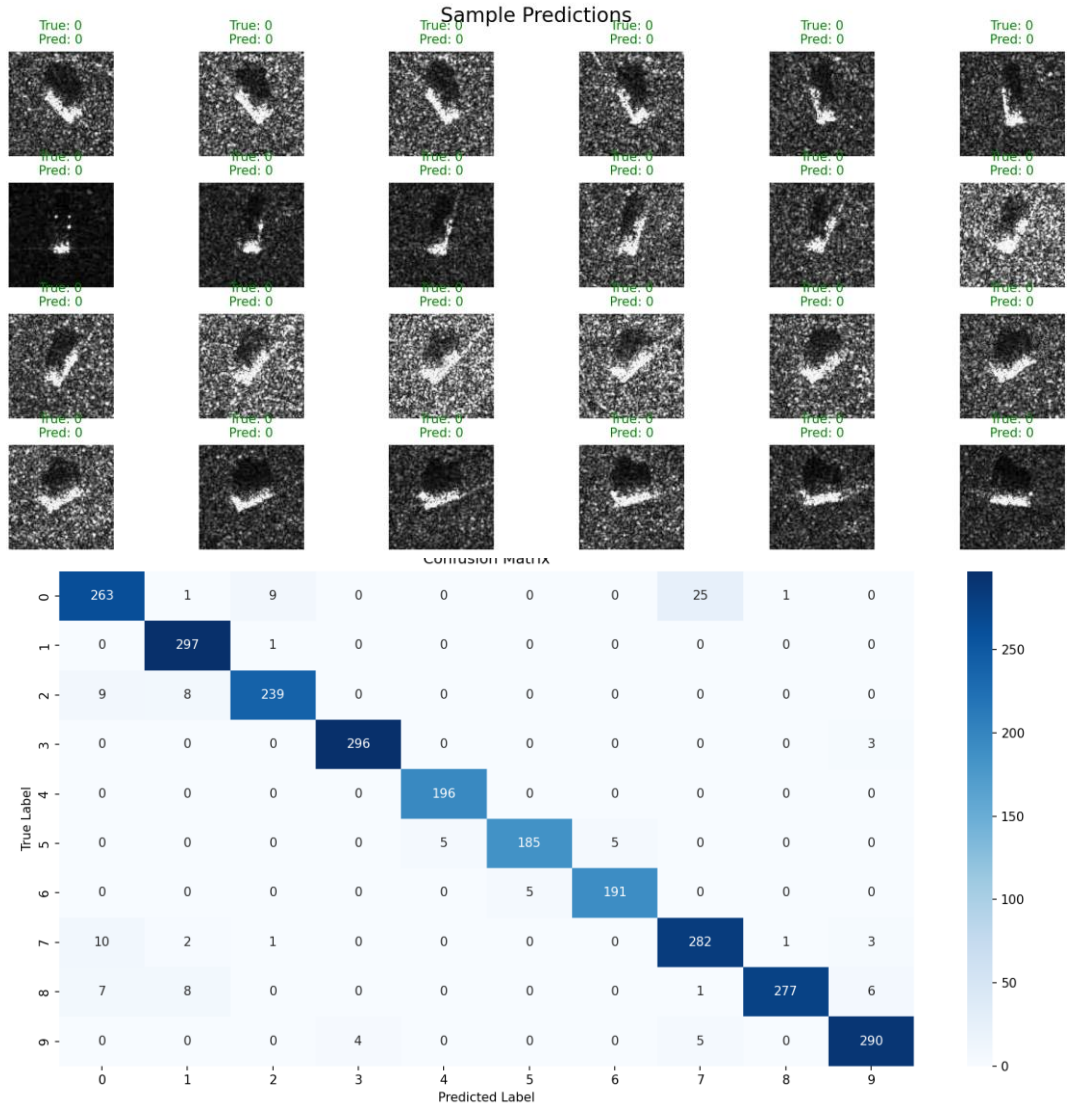
plt.tight_layout()
plt.suptitle("Sample Predictions", fontsize=16)
plt.subplots_adjust(top=0.9)
plt.show(block=True) # 阻塞显示，等待用户关闭

```

训练结果及分析

当 批大小 BATCH_SIZE = 16，训练轮数 EPOCHS = 100，学习率 LR = 5e-4 时





命令行输出如下：

```
Epoch 1/100 | Train Loss: 2.3021 | Train Acc: 0.1092 | Val Loss: 2.2992 | Val Acc: 0.1134
Epoch 2/100 | Train Loss: 2.3006 | Train Acc: 0.1116 | Val Loss: 2.2978 | Val Acc: 0.1153
Epoch 3/100 | Train Loss: 2.3000 | Train Acc: 0.1096 | Val Loss: 2.2968 | Val Acc: 0.1688
Epoch 4/100 | Train Loss: 2.2988 | Train Acc: 0.1191 | Val Loss: 2.2958 | Val Acc: 0.1654
Epoch 5/100 | Train Loss: 2.2975 | Train Acc: 0.1250 | Val Loss: 2.2949 | Val Acc: 0.1533
Epoch 6/100 | Train Loss: 2.2963 | Train Acc: 0.1380 | Val Loss: 2.2938 | Val Acc: 0.1669
Epoch 7/100 | Train Loss: 2.2960 | Train Acc: 0.1167 | Val Loss: 2.2929 | Val Acc: 0.1832
Epoch 8/100 | Train Loss: 2.2947 | Train Acc: 0.1416 | Val Loss: 2.2915 | Val Acc: 0.1916
Epoch 9/100 | Train Loss: 2.2937 | Train Acc: 0.1518 | Val Loss: 2.2899 | Val Acc: 0.1976
Epoch 10/100 | Train Loss: 2.2920 | Train Acc: 0.1494 | Val Loss: 2.2874 | Val Acc: 0.1935
Epoch 11/100 | Train Loss: 2.2896 | Train Acc: 0.1629 | Val Loss: 2.2838 | Val Acc: 0.1885
```

```
Epoch 12/100 | Train Loss: 2.2853 | Train Acc: 0.1700 | Val Loss: 2.2772 | Val Acc: 0.2079
Epoch 13/100 | Train Loss: 2.2767 | Train Acc: 0.1668 | Val Loss: 2.2632 | Val Acc: 0.2140
Epoch 14/100 | Train Loss: 2.2558 | Train Acc: 0.1751 | Val Loss: 2.2276 | Val Acc: 0.2071
Epoch 15/100 | Train Loss: 2.1952 | Train Acc: 0.1735 | Val Loss: 2.1306 | Val Acc: 0.1927
Epoch 16/100 | Train Loss: 2.0811 | Train Acc: 0.1826 | Val Loss: 2.0434 | Val Acc: 0.2295
Epoch 17/100 | Train Loss: 1.9984 | Train Acc: 0.2334 | Val Loss: 1.9446 | Val Acc: 0.2993
Epoch 18/100 | Train Loss: 1.8548 | Train Acc: 0.2729 | Val Loss: 1.7427 | Val Acc: 0.3080
Epoch 19/100 | Train Loss: 1.4848 | Train Acc: 0.3884 | Val Loss: 1.5643 | Val Acc: 0.2731
Epoch 20/100 | Train Loss: 1.2339 | Train Acc: 0.4424 | Val Loss: 1.3514 | Val Acc: 0.3877
Epoch 21/100 | Train Loss: 1.1456 | Train Acc: 0.4933 | Val Loss: 1.0295 | Val Acc: 0.5269
Epoch 22/100 | Train Loss: 1.0150 | Train Acc: 0.5481 | Val Loss: 0.9582 | Val Acc: 0.5664
```

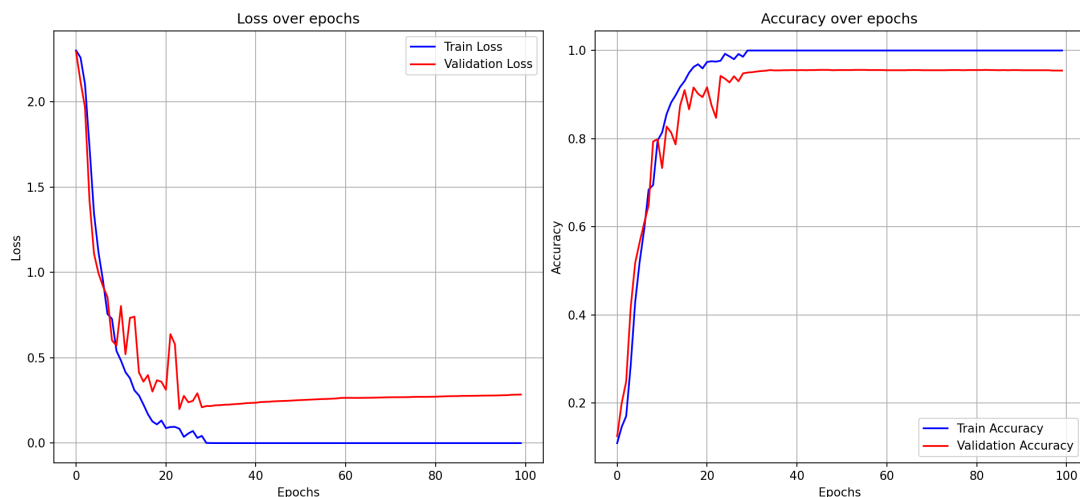
Epoch 23/100 Train Loss: 0.8832 Train Acc: 0.6199 Val Loss: 0.8126 Val Acc: 0.6506	Epoch 62/100 Train Loss: 0.0716 Train Acc: 0.9807 Val Loss: 0.4888 Val Acc: 0.8790
Epoch 24/100 Train Loss: 0.8184 Train Acc: 0.6514 Val Loss: 1.0150 Val Acc: 0.5520	Epoch 63/100 Train Loss: 0.0156 Train Acc: 0.9976 Val Loss: 0.5030 Val Acc: 0.8900
Epoch 25/100 Train Loss: 0.7505 Train Acc: 0.6838 Val Loss: 1.4864 Val Acc: 0.4241	Epoch 64/100 Train Loss: 0.0115 Train Acc: 0.9972 Val Loss: 0.5024 Val Acc: 0.8881
Epoch 26/100 Train Loss: 0.6765 Train Acc: 0.7256 Val Loss: 0.6603 Val Acc: 0.7159	Epoch 65/100 Train Loss: 0.0073 Train Acc: 0.9984 Val Loss: 0.6116 Val Acc: 0.8756
Epoch 27/100 Train Loss: 0.6148 Train Acc: 0.7610 Val Loss: 1.0258 Val Acc: 0.6498	Epoch 66/100 Train Loss: 0.0063 Train Acc: 0.9984 Val Loss: 0.9510 Val Acc: 0.8228
Epoch 28/100 Train Loss: 0.6369 Train Acc: 0.7401 Val Loss: 0.6589 Val Acc: 0.7348	Epoch 67/100 Train Loss: 0.0486 Train Acc: 0.9811 Val Loss: 0.6342 Val Acc: 0.8767
Epoch 29/100 Train Loss: 0.4736 Train Acc: 0.8115 Val Loss: 0.7581 Val Acc: 0.7178	Epoch 68/100 Train Loss: 0.0081 Train Acc: 0.9972 Val Loss: 1.0467 Val Acc: 0.8346
Epoch 30/100 Train Loss: 0.4298 Train Acc: 0.8450 Val Loss: 0.5808 Val Acc: 0.7754	Epoch 69/100 Train Loss: 0.1091 Train Acc: 0.9673 Val Loss: 0.8870 Val Acc: 0.8323
Epoch 31/100 Train Loss: 0.3781 Train Acc: 0.8498 Val Loss: 0.6465 Val Acc: 0.7587	Epoch 70/100 Train Loss: 0.0322 Train Acc: 0.9921 Val Loss: 0.6667 Val Acc: 0.8600
Epoch 32/100 Train Loss: 0.3484 Train Acc: 0.8640 Val Loss: 0.7635 Val Acc: 0.7371	Epoch 71/100 Train Loss: 0.0089 Train Acc: 0.9980 Val Loss: 0.5473 Val Acc: 0.8832
Epoch 33/100 Train Loss: 0.3495 Train Acc: 0.8707 Val Loss: 0.5529 Val Acc: 0.8065	Epoch 72/100 Train Loss: 0.0561 Train Acc: 0.9791 Val Loss: 0.5899 Val Acc: 0.8456
Epoch 34/100 Train Loss: 0.3117 Train Acc: 0.8813 Val Loss: 0.6136 Val Acc: 0.8001	Epoch 73/100 Train Loss: 0.0471 Train Acc: 0.9862 Val Loss: 0.5343 Val Acc: 0.8794
Epoch 35/100 Train Loss: 0.2568 Train Acc: 0.9081 Val Loss: 0.6788 Val Acc: 0.7656	Epoch 74/100 Train Loss: 0.0410 Train Acc: 0.9866 Val Loss: 1.0065 Val Acc: 0.8190
Epoch 36/100 Train Loss: 0.3110 Train Acc: 0.8789 Val Loss: 0.7827 Val Acc: 0.7591	Epoch 75/100 Train Loss: 0.0095 Train Acc: 0.9980 Val Loss: 0.6234 Val Acc: 0.8832
Epoch 37/100 Train Loss: 0.1887 Train Acc: 0.9302 Val Loss: 0.7956 Val Acc: 0.7932	Epoch 76/100 Train Loss: 0.0019 Train Acc: 1.0000 Val Loss: 0.6810 Val Acc: 0.8771
Epoch 38/100 Train Loss: 0.2729 Train Acc: 0.9018 Val Loss: 0.4749 Val Acc: 0.8426	Epoch 77/100 Train Loss: 0.0018 Train Acc: 1.0000 Val Loss: 0.6408 Val Acc: 0.8866
Epoch 39/100 Train Loss: 0.2075 Train Acc: 0.9294 Val Loss: 0.7526 Val Acc: 0.7633	Epoch 78/100 Train Loss: 0.0054 Train Acc: 0.9988 Val Loss: 0.6072 Val Acc: 0.8904
Epoch 40/100 Train Loss: 0.0877 Train Acc: 0.9748 Val Loss: 0.6455 Val Acc: 0.8221	Epoch 79/100 Train Loss: 0.0066 Train Acc: 0.9988 Val Loss: 0.5370 Val Acc: 0.8976
Epoch 41/100 Train Loss: 0.1756 Train Acc: 0.9361 Val Loss: 0.4853 Val Acc: 0.8532	Epoch 80/100 Train Loss: 0.0023 Train Acc: 0.9996 Val Loss: 0.5824 Val Acc: 0.8900
Epoch 42/100 Train Loss: 0.0763 Train Acc: 0.9756 Val Loss: 0.5712 Val Acc: 0.8467	Epoch 81/100 Train Loss: 0.0010 Train Acc: 1.0000 Val Loss: 0.6957 Val Acc: 0.8801
Epoch 43/100 Train Loss: 0.0932 Train Acc: 0.9696 Val Loss: 0.6122 Val Acc: 0.8448	Epoch 82/100 Train Loss: 0.0007 Train Acc: 1.0000 Val Loss: 0.6179 Val Acc: 0.8907
Epoch 44/100 Train Loss: 0.1398 Train Acc: 0.9554 Val Loss: 0.5223 Val Acc: 0.8608	Epoch 83/100 Train Loss: 0.0122 Train Acc: 0.9965 Val Loss: 1.1702 Val Acc: 0.8179
Epoch 45/100 Train Loss: 0.0656 Train Acc: 0.9787 Val Loss: 0.7040 Val Acc: 0.8388	Epoch 84/100 Train Loss: 0.1187 Train Acc: 0.9653 Val Loss: 0.4856 Val Acc: 0.8722
Epoch 46/100 Train Loss: 0.0707 Train Acc: 0.9771 Val Loss: 0.5388 Val Acc: 0.8593	Epoch 85/100 Train Loss: 0.0343 Train Acc: 0.9894 Val Loss: 0.4694 Val Acc: 0.8938
Epoch 47/100 Train Loss: 0.0377 Train Acc: 0.9862 Val Loss: 0.5845 Val Acc: 0.8589	Epoch 86/100 Train Loss: 0.0653 Train Acc: 0.9763 Val Loss: 0.4653 Val Acc: 0.8888
Epoch 48/100 Train Loss: 0.0139 Train Acc: 0.9968 Val Loss: 0.6857 Val Acc: 0.8555	Epoch 87/100 Train Loss: 0.0039 Train Acc: 0.9996 Val Loss: 0.5717 Val Acc: 0.8862
Epoch 49/100 Train Loss: 0.0476 Train Acc: 0.9823 Val Loss: 0.5392 Val Acc: 0.8649	Epoch 88/100 Train Loss: 0.0024 Train Acc: 0.9996 Val Loss: 0.5538 Val Acc: 0.8866
Epoch 50/100 Train Loss: 0.0987 Train Acc: 0.9677 Val Loss: 1.1536 Val Acc: 0.7420	Epoch 89/100 Train Loss: 0.0014 Train Acc: 1.0000 Val Loss: 0.5407 Val Acc: 0.8938
Epoch 51/100 Train Loss: 0.1995 Train Acc: 0.9353 Val Loss: 0.8233 Val Acc: 0.8031	Epoch 90/100 Train Loss: 0.0033 Train Acc: 0.9988 Val Loss: 0.8021 Val Acc: 0.8634
Epoch 52/100 Train Loss: 0.0578 Train Acc: 0.9826 Val Loss: 0.4411 Val Acc: 0.8759	Epoch 91/100 Train Loss: 0.0018 Train Acc: 1.0000 Val Loss: 0.6042 Val Acc: 0.8919
Epoch 53/100 Train Loss: 0.0312 Train Acc: 0.9917 Val Loss: 0.6283 Val Acc: 0.8471	Epoch 92/100 Train Loss: 0.0009 Train Acc: 1.0000 Val Loss: 0.6526 Val Acc: 0.8839
Epoch 54/100 Train Loss: 0.0207 Train Acc: 0.9941 Val Loss: 0.9469 Val Acc: 0.8084	Epoch 93/100 Train Loss: 0.0009 Train Acc: 0.9996 Val Loss: 0.8315 Val Acc: 0.8687
Epoch 55/100 Train Loss: 0.0700 Train Acc: 0.9752 Val Loss: 0.6069 Val Acc: 0.8585	Epoch 94/100 Train Loss: 0.0013 Train Acc: 1.0000 Val Loss: 0.6970 Val Acc: 0.8851
Epoch 56/100 Train Loss: 0.0600 Train Acc: 0.9807 Val Loss: 0.8693 Val Acc: 0.8247	Epoch 95/100 Train Loss: 0.0009 Train Acc: 1.0000 Val Loss: 0.6563 Val Acc: 0.8911
Epoch 57/100 Train Loss: 0.0568 Train Acc: 0.9815 Val Loss: 0.7540 Val Acc: 0.8471	Epoch 96/100 Train Loss: 0.0005 Train Acc: 1.0000 Val Loss: 0.6732 Val Acc: 0.8888
Epoch 58/100 Train Loss: 0.0490 Train Acc: 0.9842 Val Loss: 0.5642 Val Acc: 0.8767	Epoch 97/100 Train Loss: 0.0005 Train Acc: 1.0000 Val Loss: 0.6589 Val Acc: 0.8892
Epoch 59/100 Train Loss: 0.0063 Train Acc: 0.9992 Val Loss: 0.6388 Val Acc: 0.8665	Epoch 98/100 Train Loss: 0.0338 Train Acc: 0.9862 Val Loss: 0.7224 Val Acc: 0.8710
Epoch 60/100 Train Loss: 0.0024 Train Acc: 1.0000 Val Loss: 0.6043 Val Acc: 0.8759	Epoch 99/100 Train Loss: 0.0703 Train Acc: 0.9811 Val Loss: 0.4792 Val Acc: 0.8949
Epoch 61/100 Train Loss: 0.0764 Train Acc: 0.9771 Val Loss: 1.3607 Val Acc: 0.7348	Epoch 100/100 Train Loss: 0.0138 Train Acc: 0.9961 Val Loss: 0.7176 Val Acc: 0.8619

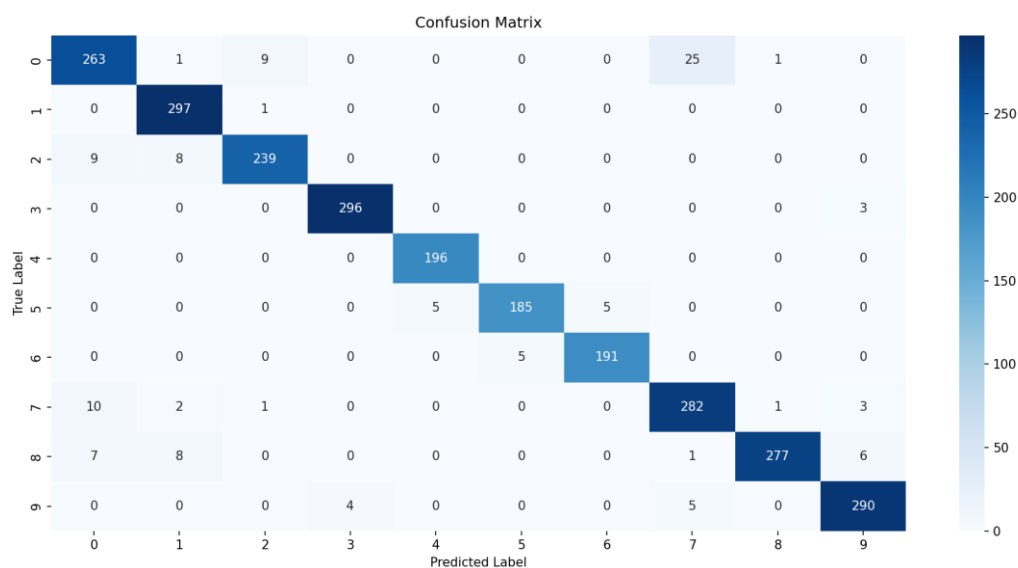
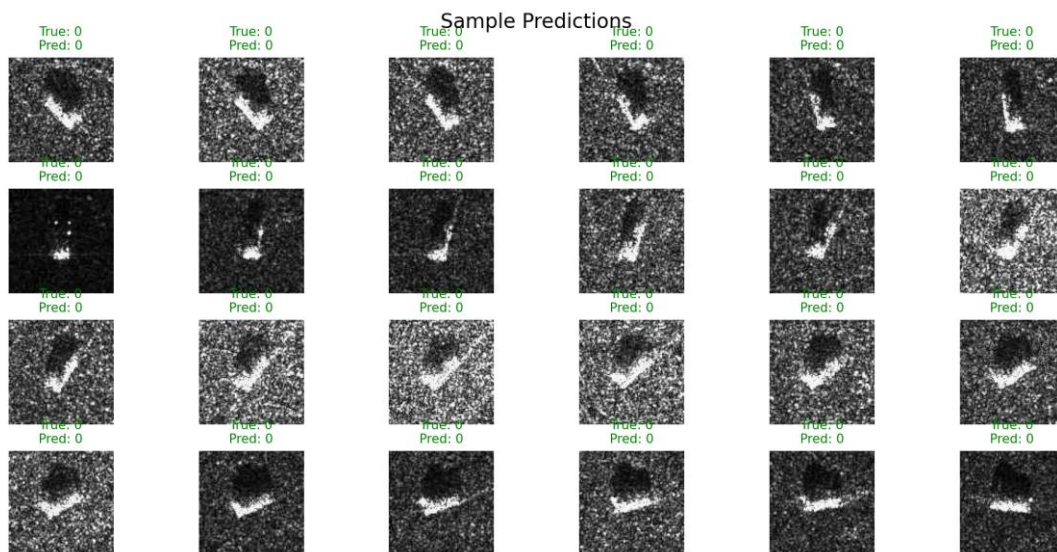
这次训练在训练集上收敛得非常快：训练损失从初始的约 2.30 迅速下降并在后期接近零，训练准确率也稳定爬升到接近 100%，说明模型在记住训练样本方面表现极好。验证集的表现则波动较大：验证损失在训练早期下降明显，但中后期出现反复上升和下降，最终稳定在比训练损失高得多的区间；验证准确率总体在 0.74 - 0.90 之间波动，最终多次徘徊在约 0.86 - 0.89，这表明模型在训练集上过拟合明显，泛化能力有限但仍能达到较高的验证准确率峰值。**训练准确率接近 100% 但验证准确率波动且远低于训练**，这是典型的**过拟合**或训练/损失计算不匹配的表现

从混淆矩阵和样本预测可以看出，绝大多数类别的主对角线值很高，说明模型对多数类能做出正确判断，但仍存在若干系统性混淆：例如类 0 会被误判为 2 和 7，类 2 有部分被判为 0 或 1，类 5 对 4 和 6 有少量混淆，类 9 有少量被判为 2 或 7，类 8 与 5、7 也有零星错误。样本展示中某些类（如 0）在可视样本上几乎全部预测正确，说明在某些类别上数据特征明显且模型拟合良好；但混淆分布提示部分类别间特征区分度不足或样本分布不均。

于是，我适当增加了学习率，再看训练效果。

批大小 BATCH_SIZE = 16，训练轮数 EPOCHS = 100，学习率 LR = $5e-3$ 时





把学习率从 $5e-4$ 提高到 $5e-3$ 后，训练过程变得极其迅速但也更不稳定。日志显示训练损失在前几十个 epoch 就急速下降并接近零，训练准确率几乎达到 100%，这说明模型在训练集上几乎完全拟合；与此同时验证损失虽然在早期下降，但中后期出现明显的波动和多次上升，验证准确率在 $0.74 - 0.90$ 之间来回震荡并最终多次徘徊在约 $0.86 - 0.89$ ，说明泛化性能并没有随训练集性能的提升而稳定提高，反而出现了过拟合和训练-验证不一致的迹象。

4.1 MSTAR 改进 CNN 分类

针对原始模型的缺点，我优化了模型，首先，模型本身从原来的 ConvNet 升级成带 BatchNorm 的 EnhancedConvNet，并且全连接层的维度也做了合理压缩，减少过拟合，同时把 LogSoftmax 去掉，改用 CrossEntropyLoss 直接处理 logits，结构更干净^[20]。

并且训练流程也不再是一直跑到底，而是加入了早停机制、学习率调度器、权重衰减，让训练过程能自动调整节奏，避免过拟合或学习率卡住。同时加入了旋转、模糊^[21]、色彩抖动、随机裁剪等多种扰动，让模型学到更强的泛化能力。推理阶段也加入了 TTA（测试时增强），通过多种变换取平均预测，提高最终验证准确率^[22]。

```
# ===== 增强的训练函数（带早停+学习率调度）
=====
def train(model, train_loader, criterion, optimizer, epochs,
validation_loader=None,
          scheduler=None, patience=15, save_path="best_model.pth"):
    train_losses = []
    train_accs = []
    val_losses = []
    val_accs = []

    best_val_acc = 0.0
    best_epoch = 0
    patience_counter = 0 # 早停计数器

    for epoch in range(epochs):
        # 训练阶段
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
```

```

optimizer.step()

running_loss += loss.item() * images.size(0)
_, predicted = torch.max(outputs, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()

epoch_loss = running_loss / len(train_loader.dataset)
epoch_acc = correct / total
train_losses.append(epoch_loss)
train_accs.append(epoch_acc)

# 验证阶段
val_loss = 0.0
val_correct = 0
val_total = 0
if validation_loader:
    model.eval()
    with torch.no_grad():
        for images, labels in validation_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item() * images.size(0)
            _, predicted = torch.max(outputs, 1)
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

val_loss = val_loss / len(validation_loader.dataset)
val_acc = val_correct / val_total
val_losses.append(val_loss)
val_accs.append(val_acc)

# 学习率调度
if scheduler is not None:
    if isinstance(scheduler,
optim.lr_scheduler.ReduceLROnPlateau):
        scheduler.step(val_acc)
    else:
        scheduler.step()

# 早停逻辑 + 保存最优模型
if val_acc > best_val_acc:
    best_val_acc = val_acc
    best_epoch = epoch

```

```

        patience_counter = 0
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'val_acc': val_acc,
        }, save_path)
        print(f"✅ 保存最优模型: Epoch {epoch+1}, Val Acc:
{val_acc:.4f}")
    else:
        patience_counter += 1

    # 打印日志（获取当前学习率，替代 verbose）
    current_lr = optimizer.param_groups[0]['lr']
    print(f"Epoch {epoch+1}/{epochs} | LR: {current_lr:.6f} | "
          f"Train Loss: {epoch_loss:.4f} | Train Acc:
{epoch_acc:.4f} | "
          f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f} | "
          f"Patience: {patience_counter}/{patience}")

    # 触发早停
    if patience_counter >= patience:
        print(f"\n⚠️ 早停触发! 最优 Epoch: {best_epoch+1}, 最优 Val
Acc: {best_val_acc:.4f}")
        break
    else:
        print(f"Epoch {epoch+1}/{epochs} | "
              f"Train Loss: {epoch_loss:.4f} | Train Acc:
{epoch_acc:.4f}")

    # 加载最优模型
    if validation_loader:
        checkpoint = torch.load(save_path)
        model.load_state_dict(checkpoint['model_state_dict'])
        print(f"\n🏆 训练完成，加载最优模型 (Epoch {checkpoint['epoch']+1},
Val Acc: {checkpoint['val_acc']:.4f}) ")

    return train_losses, train_accs, val_losses, val_accs

# ===== TTA 测试增强函数 =====
def tta_predict(model, image, tta_transforms, device):
    """测试时增强：多变换后取平均预测"""
    model.eval()
    preds = []
    with torch.no_grad():

```

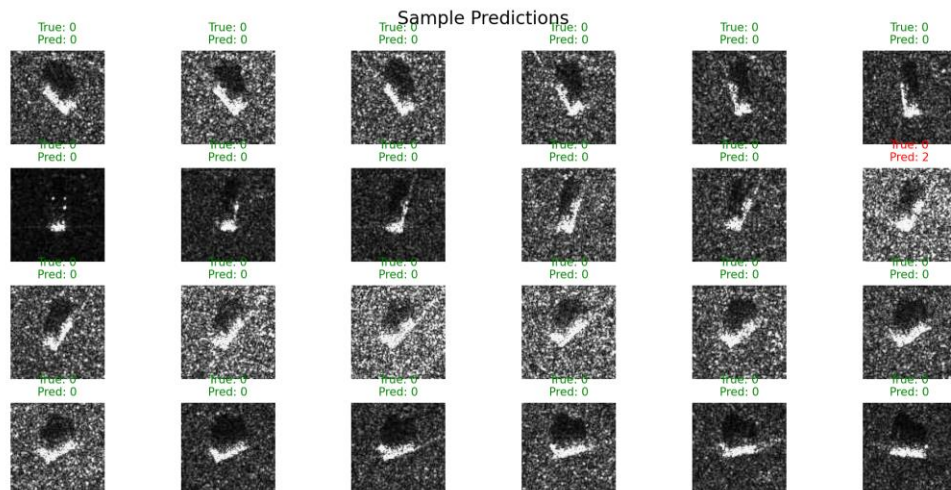
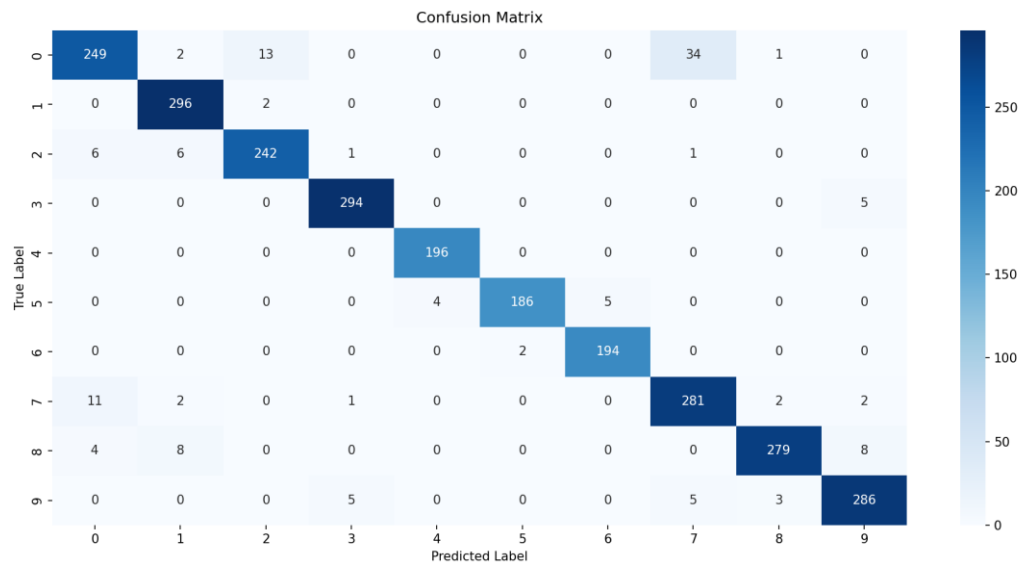
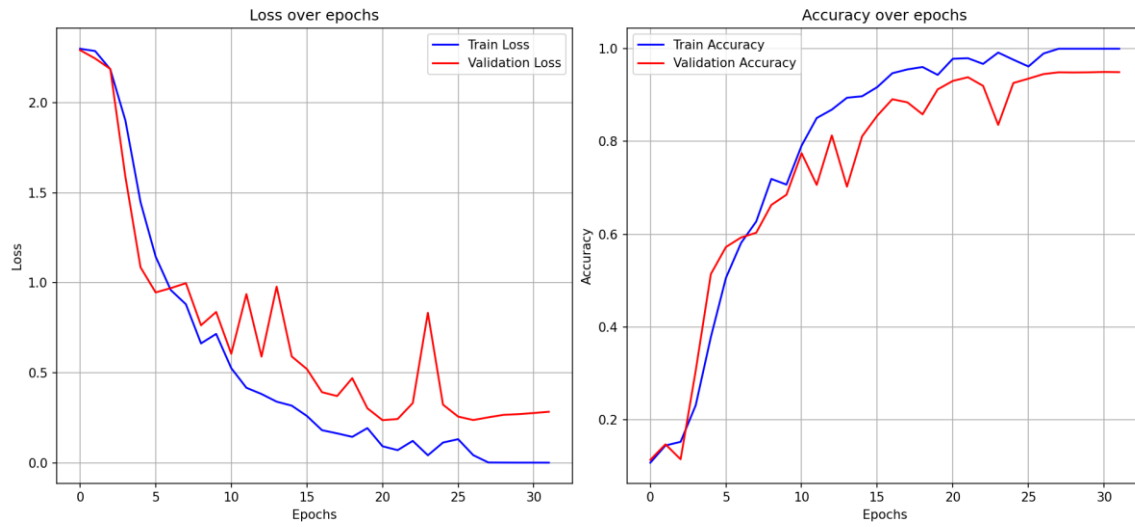
```

    for transform in tta_transforms:
        # 应用变换
        img_tensor = transform(image).unsqueeze(0).to(device)
        output = model(img_tensor)
        pred = torch.softmax(output, dim=1)
        preds.append(pred)
    # 取平均后返回预测结果
    avg_pred = torch.mean(torch.stack(preds), dim=0)
    return torch.argmax(avg_pred).item(), avg_pred

# ===== 定义 TTA 变换（测试增强） =====
tta_transforms = [
    # 变换 1: 原图
    transforms.Compose([
        transforms.Resize(IMAGE_SIZE),
        transforms.Grayscale(num_output_channels=3),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
    ]),
    # 变换 2: 水平翻转
    transforms.Compose([
        transforms.Resize(IMAGE_SIZE),
        transforms.RandomHorizontalFlip(p=1.0),
        transforms.Grayscale(num_output_channels=3),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
    ]),
    # 变换 3: 小角度旋转
    transforms.Compose([
        transforms.Resize(IMAGE_SIZE),
        transforms.RandomRotation(degrees=10),
        transforms.Grayscale(num_output_channels=3),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
    ])
]

```

使用了改进的模型后, $LR = 5e-3$, $EPOCHS = 32$ 时系统自动判定已经训练完成。



学习率为 $5e-3$ 、训练轮数设为 32 时，训练损失在前半程迅速下降并在后期接近零，训练准确率很快攀升到接近 100%，而验证集在第十几到二十几轮出现明显的跃升——从 0.57 一路上升到约 0.93 - 0.95 并在 28 - 32 轮附近稳定，最终验证损失在约 0.28 附近徘徊，验证准确率收敛到约 **94.9%**。总体来看，模型在较少轮数内就达到了很高的验证精度，验证曲线比之前长轮次训练时的剧烈震荡要平滑得多，但训练集与验证集之间仍存在差距（训练几乎完全拟合，验证略低），说明模型容量很大且学习速度很快，同时通过较短训练周期避免了更严重的过拟合。

结论

在相同批大小和数据条件下，把学习率从 $5e-4$ 提升到 $5e-3$ 并把训练轮数控制在 32 次后，模型在训练集上收敛得极快、训练损失几乎归零、训练精度接近 100%，验证集也在较少的 epoch 内跃升并稳定在约 94 - 95% 的高水平（验证损失约 0.28），说明更大学习率配合较短训练周期能在速度和验证精度上取得明显收益；但同时要注意，这种配置仍保留训练—验证间的差距，提示模型容量大且存在记忆训练集细节的倾向。与较小学习率时的表现相比，低学习率下收敛更平滑、验证曲线抖动更小但收敛慢、最终验证稳定性有时更好；而高学习率则更容易快速到达低损失区但也更容易在损失面上震荡或过拟合，因而需要配合早停、学习率衰减或更强的正则化来稳固泛化。

结构上的改进（例如去掉多余的 LogSoftmax 或改用与之匹配的损失、合理设置 dropout、确认全连接层输入维度、以及必要时缩减或正则化全连接层）也能直接提升训练稳定性并减少信息丢失，这也是本次在较大学习率下仍能获得较好验证效果的重要原因。

5. 参考文献

- [24] S. Brusch, S. Lehner, T. Fritz, M. Soccorsi, A. Soloviev, and B. van Schie, "Ship surveillance with TerraSAR-X," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 3, pp. 1092-1103, Mar. 2011.
- [25] H. Lang, J. Zhang, X. Zhang, and J. Meng, "Ship classification in SAR image by joint feature and classifier selection," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 2, pp. 212-216, Feb. 2016.
- [26] H. Lang, S. Wu, and Y. Xu, "Ship classification in SAR images improved by AIS knowledge transfer," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 3, pp. 439-443, Mar. 2018.
- [27] M. Jiang, X. Yang, Z. Dong, S. Fang, and J. Meng, "Ship classification based on superstructure scattering features in SAR images," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 5, pp. 616-620, May 2016.
- [28] W.-T. Chen, K.-F. Ji, X.-W. Xing, H.-X. Zou, and H. Sun, "Ship recognition in high resolution SAR imagery based on feature selection," in *Proc. Int. Conf. Comput. Vis. Remote Sens.*, Dec. 2012, pp. 301-305.
- [29] Y. Wu et al., "Joint convolutional neural network for small-scale ship classification in SAR images," in *Proc. IGARSS-IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2019, pp. 2619-2622.
- [30] Z. Cui, Q. Li, Z. Cao, and N. Liu, "Dense attention pyramid networks for multi-scale ship detection in SAR images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 11, pp. 8983-8997, Nov. 2019.
- [31] J. Ding, B. Chen, H. Liu, and M. Huang, "Convolutional neural network with data augmentation for SAR target recognition," *IEEE Geosci. Remote Sens. Lett.*, vol. 13,

no. 3, pp. 364-368, Mar. 2016.

- [32] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in Proc. Adv. Neural Inf. Process. Syst., 2017, pp. 4077–4087.
- [33] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Jun. 2018, pp. 1199–1208.
- [34] O. Vinyals, C. Blundell, T. Lillicrap, and D. Wierstra, "Matching networks for one shot learning," in Proc. Adv. Neural Inf. Process. Syst., 2016, pp. 3630–3638.
- [35] X. Leng, K. Ji, S. Zhou, X. Xing, and H. Zou, "Discriminating ship from radio frequency interference based on noncircularity and non-Gaussianity in Sentinel-1 SAR imagery," IEEE Trans. Geosci. Remote Sens., vol. 57, no. 1, pp. 352–363, Jan. 2019.
- [36] X. Leng, K. Ji, S. Zhou, and X. Xing, "Ship detection based on complex signal kurtosis in single-channel SAR imagery," IEEE Trans. Geosci. Remote Sens., vol. 57, no. 9, pp. 6447-6461, Sep. 2019.
- [37] Li, J.G., Zhang, R. and Li, Y, "Multiscale convolutional neural network for the detection of built-up areas in highresolution SAR images," in Geoscience and Remote Sensing Symposium IEEE, 910 –913 (2016).
- [38] Profeta, A., Rodriguez, A. and Clouse, H. S, "Convolutional neural networks for synthetic aperture radar classification," in Proc. SPIE 9843,98430M, 1 –10 (2016).
- [39] Chen, S.Z., Wang H.P. and Xu, F, "Target classification using the deep convolutional networks for SAR images," in IEEE Transactions on Geoscience & Remote Sensing, 4806 –4817 (2016).
- [40] Ding, J., Chen, B. and Liu H, W., "Convolutional neural network with data

augmentation for SAR target recognition,” in IEEE Geoscience & Remote Sensing Letters, 364 –368 (2016).

- [41] Srivastava, N., Hinton, G. and Krizhevsky, A, “Dropout: A simple way to prevent neural networks from overfitting,” J. Mach. Learn. Res, 15 (1), 1929 –1958 (2014).
- [42] Glorot, X. and Bengio, Y, “Understanding the difficulty of training deep feedforward neural networks,” J. Mach. Learn. Res, 9 249 –256 (2010).
- [43] S. Wagner, K. Barth, and S. Brggenwirth, “A deep learning SAR ATR system using regularization and prioritized classes”, Proc. of 2010 2017 IEEE Radar Conference (RadarConf), 2017, pp. 0772 0777
- [44] S.Deng,L.Du,C.Li,J.Ding,andH.Liu,“SARAAutomaticTargetRecognitionBasedonEuclidean Distance Restricted Autoencoder”, IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing,vol. 10, no. 7, 2017,
- [45] E. R. Keydel, S. W. Lee, and J. T. Moore, “MSTAR extended operating conditions: A tutorial”, in Proc. of 3rd SPIE Conf. Algorithms SAR Imagerytext, 1996, vol. 2757, pp. 228242.

附件：设计程序